

Institut de Formation d'Assurances et de Gestion
Établissement Privé de Formation Supérieure
Agréé par le Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

MEMOIRE DE FIN D'ÉTUDES

Pour l'obtention du diplôme de

LICENCE

Filière : Informatique

Spécialité : Métiers de l'Informatique

THEME

**Smart Electric Vehicle Charging Station
Management Platform:
“EV Charge Manager”**

Réalisé Par :

- Yacine Omar FODHIL
- Abderahmane Mohamed El
Hachemi Chibani

Encadré Par :

- Mme Nadia BENAHMED-EL
ALIA
- M. Amar BALLA

Dedication

To our parents, who sacrificed so much and, at times, everything, so that we could reach this level. Your unwavering love, endless patience, and constant support were the foundation upon which we built our journey. You were our light in moments of doubt and our strength when the path felt difficult.

To our brothers and sisters, whose daily presence, encouragement, and belief in us made every challenge feel lighter. Your words, smiles, and support were a quiet but powerful source of motivation.

To ourselves, for our sincere collaboration, our discipline, and our commitment throughout this project. Through moments of pressure, late nights, and shared efforts, we proved that perseverance and teamwork lead to achievement.

To all our friends and classmates, with whom we shared not only knowledge but also unforgettable moments, laughter, and challenges. This academic journey would not have been the same without you, and these memories will remain with us far beyond this project.

Acknowledgments

First and foremost, we thank Allah, the Almighty, for granting us the strength, patience, and perseverance to complete this work. Without His will, none of this would have been possible.

We would also like to express our deepest gratitude to our parents and family for their sacrifices, unconditional support, constant encouragement, and unwavering belief in us throughout our journey. Their presence has been a true source of motivation and comfort.

We extend our heartfelt appreciation to our teachers, whose guidance has been invaluable:

To Mr. Balla, who was more than a teacher to us he was like a father. His wisdom, constant support, and sincere care guided us not only academically but also personally. He believed in us, encouraged us in difficult moments, and inspired us to always strive for better.

To Mrs. BENAHMED-EL ALIA, who embodied the role of a caring and guiding mother. Her kindness, patience, and dedication created a supportive environment where we felt encouraged to grow, learn, and express ourselves with confidence.

To Mr. Mokeddem, who provided us with a strong foundation in understanding development. His clarity, guidance, and generosity in sharing knowledge helped us build solid academic skills and a deeper way of thinking.

We would also like to express our sincere thanks to Mr. Chagroune for his warm welcome at SAIEG, his honest guidance, and the valuable information he generously shared with us.

We are truly grateful for their dedication, support, and the lasting impact they have had on our journey.

Abstract

Résumé (Français)

EV Charge Manager est une plateforme web multi-tenant intégrée dédiée à la gestion des infrastructures de recharge pour véhicules électriques en Algérie. Face à l'expansion progressive de la mobilité électrique et à l'absence de solutions logicielles adaptées au contexte algérien, ce projet vise à fournir un système complet couvrant l'ensemble du cycle opérationnel d'un réseau de bornes de recharge.

La plateforme offre une interface unifiée de supervision du réseau, de gestion des sessions de recharge, de traitement de la facturation et de suivi des tickets de maintenance. Elle communique avec les équipements physiques via le protocole OCPP 1.6 et s'appuie sur une architecture trois tiers : une application web React 18/TypeScript, une API REST Node.js/Express.js et une base de données MySQL 8. Le modèle multi-tenant garantit une isolation complète des données entre opérateurs indépendants, avec un contrôle d'accès granulaire fondé sur les rôles (RBAC).

La plateforme est complétée par des applications mobiles dédiées, offrant la localisation des stations, la navigation GPS, la gestion des sessions et le suivi des interventions de maintenance sur le terrain, avec des notifications en temps réel via Firebase Cloud Messaging.

Les résultats obtenus valident la conception d'une solution de gestion de bornes opérationnelle et adaptée au marché algérien, constituant une base technique solide pour un déploiement à grande échelle.

Mots-clés : Véhicule électrique, Borne de recharge, OCPP 1.6 , ISO 15118.

Abstract (English)

EV Charge Manager is an integrated multi-tenant web platform dedicated to the management of electric vehicle charging infrastructure in Algeria. In response to the progressive expansion of electric mobility and the absence of software solutions adapted to the Algerian context, this project aims to provide a complete system covering the entire operational cycle of a charging network.

The platform offers a unified interface for network supervision, charging session management, billing processing, and maintenance ticket tracking. It communicates with physical hardware using the OCPP 1.6 protocol and is built on a three-tier architecture: a React 18/TypeScript web application, a Node.js/Express.js REST API, and a MySQL 8 database. The multi-tenant model guarantees complete data isolation between independent operators, with granular role-based access control (RBAC).

The platform is complemented by dedicated mobile applications, providing station localisation, GPS navigation, session management, and field maintenance tracking, with real-time notifications via Firebase Cloud Messaging.

The results obtained validate the design of an operational charging management solution adapted to the Algerian market, establishing a solid technical foundation for large-scale deployment.

Keywords: Electric vehicle , Charging station , OCPP 1.6 , ISO 15118.

ملخص (بالعربية)

EV Charge Manager هي منصة ويب متكاملة متعددة المستأجرين مخصصة لإدارة البنية التحتية لشحن المركبات الكهربائية في الجزائر. في ظل التوسع التدريجي للتنقل الكهربائي وغياب الحلول البرمجية الملائمة للسياق الجزائري، يهدف هذا المشروع إلى توفير نظام شامل يغطي الدورة التشغيلية الكاملة لشبكة محطات الشحن. تُقدّم المنصة واجهة موحدة للإشراف على الشبكة، وإدارة جلسات الشحن، ومعالجة الفواتير، وتتبع تذاكر الصيانة. وهي تتواصل مع الأجهزة الفيزيائية عبر بروتوكول OCPP 1.6 وتعتمد معمارية ثلاثية الطبقات: تطبيق ويب React 18/TypeScript، وواجهة برمجية REST API مبنية بـ Node.js/Express.js، وقاعدة بيانات MySQL 8. يتضمن النموذج متعدد المستأجرين عزلاً كاملاً للبيانات بين المشغلين المستقلين، مع تحكم دقيق في الوصول مبني على الأدوار (RBAC).

تُكَمّل المنصة بتطبيقات مخصصة للهاتف المحمول، توفر تحديد موقع المحطات، والملاحة عبر نظام GPS، وإدارة الجلسات، ومتابعة تدخلات الصيانة الميدانية، مع إشعارات فورية عبر Firebase Cloud Messaging. تُثبت النتائج المحققة صحة تصميم حل إدارة محطات الشحن العملي والمنكّيف مع السوق الجزائرية، مما يُشكّل أساساً تقنياً متيناً لنشره على نطاق واسع.

الكلمات المفتاحية: المركبة الكهربائية، محطة الشحن، OCPP 1.6، ISO 15118.

Contents

Dedication	i
Acknowledgments	ii
Abstract	iii
List of Abbreviations	xiii
General Introduction	1
State of the Art	6
1 ISO 15118 Standard : Definition and Relevance to Smart EV Charging	6
1.1 What is ISO 15118?	6
1.2 Why ISO 15118 Matters for Smart EV Charging	7
1.3 Reference Implementation	7
1.4 System Architecture of the Reference Implementation	7
1.5 DC Charging Mode and CC-CV Strategy	8
1.6 Dynamic Parameter Exchange	8
1.7 Demand Response with Photovoltaic Integration	8
1.8 IoT Integration and Monitoring	9
1.9 Security	9
1.10 Summary	9
1.11 Conclusion	9
2 OCPP 1.6 vs. OCPP 2.0 : A Comprehensive Comparison	11
2.1 Device Management and Structure	11
2.2 Smart Charging Capabilities	11
2.3 Message Handling and Communication	12
2.4 Enhanced Security	12
2.5 Firmware Management	13
2.6 Backward Compatibility	13

2.7	Use Cases and Scalability	13
2.8	Technical Comparison	14
2.9	Why OCPP 1.6 Was Chosen for This Project	14
2.9.1	Limited EV Infrastructure in Algeria	14
2.9.2	Hardware Availability	15
2.9.3	Simplicity for Academic Prototype	15
2.9.4	Network Constraints	15
2.9.5	Industry Compatibility	15
2.10	Overview of Existing EV Charging Management Platforms	15
2.10.1	PlugShare and Driivz	16
2.10.2	ChargePoint and ChargeLab	16
2.10.3	EV Connect and Tap Electric	16
2.10.4	Positioning of EV Charge DZ	16
2.11	Conclusion	17
	Design and Implementation	19
3	Requirements Analysis and Specification	19
3.1	Identification of System Actors	19
3.2	Functional Requirements	19
3.3	Non-Functional Requirements	20
3.4	Use Case Diagram	21
3.5	Technical Constraints	21
3.5.1	Protocol Constraint — OCPP 1.6	21
3.5.2	Regulatory and Market Constraints	21
3.5.3	Network Infrastructure Constraints	21
3.6	Conclusion	24
4	EV Charge Manager : Architecture and Design	25
4.1	General Architecture	25
4.2	Technology Stack	26
4.3	Multi-Tenant Architecture	26
4.4	Data Model	26
4.5	User Roles and Access Control	27
4.6	Authentication Design	29
4.7	Conclusion	32
5	EV Charge Manager : Implementation and Testing	33
5.1	Development Environment	33

5.2	Authentication Interface	33
5.3	Dashboard Module	34
5.4	Station Management Module	35
5.4.1	Connector Standards	35
5.4.2	Station Status Values	36
5.4.3	Station Actions	36
5.5	Session Monitoring Module	36
5.6	Billing Module	38
5.7	Maintenance Ticket System	38
5.8	User Management Module	39
5.9	Push Notifications Module	39
5.10	Data Resilience and Offline Mode	42
5.11	Platform Summary	42
5.12	Conclusion	42
6	EVCharge : The Mobile Companion Application	44
6.1	Motivation and Objectives	44
6.2	Technology Stack	44
6.3	Application Architecture	45
6.4	App-Dashboard Integration	46
6.4.1	Shared Backend, Separated API Routes	46
6.4.2	Authentication Difference	46
6.5	Interactive Station Map	46
6.6	Navigation with OpenRouteService	47
6.6.1	The ORS Proxy Architecture	49
6.6.2	Route Parsing and Display	49
6.7	Real-Time Station Updates via WebSocket	49
6.8	Firebase Cloud Messaging	52
6.8.1	Token Registration Flow	52
6.8.2	Notification Types and Targeting	52
6.9	Charging Session Management	55
6.10	Problem Reporting	55
6.11	Application Screens Overview	58
6.12	Conclusion	60
	General Conclusion and Perspectives	61
	References	64
A	Combined Use Case Diagram (All Actors)	66

List of Figures

3.1	Use Case Diagram — Super Admin	22
3.2	Use Case Diagram — Tenant Admin	23
3.3	Use Case Diagram — Technician	23
4.1	Class Diagram EV Charge DZ (Database-Oriented)	28
4.2	Sequence Diagram Authentication and OTP Verification Flow (Super Admin) .	31
5.1	Login Page — Credential Entry and Tenant Selection	34
5.2	Login — Super Admin Access Code Field	34
5.3	Two-Factor Verification — OTP Entry Step	34
5.4	Dashboard Module Real-Time KPI Overview and Trend Charts	35
5.5	Station Management — Table View with Status and City Filters	36
5.6	Station Management Interactive Map View (Leaflet/OpenStreetMap)	37
5.7	Session Monitoring Module Chronological Charging Session Log	37
5.8	Billing Module Payment Method Statistics and Invoice Records	38
5.9	Activity Diagram Maintenance Ticket Lifecycle	40
5.10	Maintenance Ticket Module Ticket List and New Ticket Creation Form	41
5.11	My Tasks Technician Personal Work Queue	41
5.12	User Management Module User List and Account Creation Form	41
5.13	Push Notifications Module Broadcast Composer and Notification History	42
6.1	EVCharge — Interactive Station Map interface	48
6.2	NavigationScreen — ORS route polyline and step-by-step instruction panel . .	50
6.3	Sequence Diagram — Navigation with OpenRouteService Proxy	51
6.4	NotificationsScreen — received FCM push notifications listed chronologically	53
6.5	Sequence Diagram — Firebase Cloud Messaging (Token Registration & Notifi- cation Dispatch)	54
6.6	Payment method selection — CIB Card, BaridiMob, and RFID Card options . .	56
6.7	ReportProblemSheet — fault category selection and ticket submission	57
6.8	EVCharge — Authentication screens	59
A.1	Combined Use Case Diagram — All Actors	67

B.1 Full class diagram including enumeration types	68
--	----

List of Tables

1.1	ISO 15118 Reference Implementation , Key Technical Parameters (Santos et al., 2024 [1])	9
2.1	OCPP 1.6 vs. OCPP 2.0 — Technical Feature Comparison	14
2.2	Comparison of Existing Platforms vs. EV Charge DZ	17
4.1	EV Charge Manager : Technology Stack	26
4.2	User Roles and Module Access	27
4.3	Privilege Matrix — Assigned Flags per Role Profile	29
5.1	Supported EV Connector Standards	35
5.2	EV Charge Manager Platform Specification Summary	43
6.1	EVCharge Mobile Application Technology Stack	45
6.2	Mobile App API Routes	47
6.3	FCM Notification Types	55
6.4	EVCharge Application Screen Summary	58

List of Abbreviations

API	Application Programming Interface
CC	Constant Current
CIB	Carte Interbancaire (Algerian bank card)
CPO	Charge Point Operator
CSMS	Charging Station Management System
CV	Constant Voltage
DZD	Algerian Dinar
EIM	External Identification Means
EV	Electric Vehicle
EVCC	Electric Vehicle Communication Controller
EVSE	Electric Vehicle Supply Equipment
HMI	Human-Machine Interface
HTTP	Hypertext Transfer Protocol
IEC	International Electrotechnical Commission
IoT	Internet of Things
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
JWT	JSON Web Token
KPI	Key Performance Indicator
OCPP	Open Charge Point Protocol
OTP	One-Time Password
PnC	Plug & Charge
PV	Photovoltaic
RBAC	Role-Based Access Control
REST	Representational State Transfer
RFID	Radio-Frequency Identification
RISE V2G	Robust Implementation and Standardized Extensions for V2G
SaaS	Software as a Service
SECC	Supply Equipment Communication Controller

SLA	Service Level Agreement
SoC	State of Charge
SPA	Single-Page Application
TLS	Transport Layer Security
V2G	Vehicle-to-Grid

General Introduction

1. Context

The global automotive sector is undergoing a profound transformation driven by the rapid expansion of electric vehicles (EV). According to the International Energy Agency (IEA), the global EV fleet surpassed 40 million units in 2023, with sustained growth projected throughout the decade. This transformation demands not only physical charging hardware but also sophisticated software platforms capable of managing stations, monitoring sessions, processing payments, and coordinating maintenance operations at scale.

In Algeria, this transition is gaining momentum. The country recorded 8,951 electric vehicles sold in 2025, with a national target of 1.2 million electrified vehicles by 2030, representing 10 % of the total vehicle fleet. National operators, foremost among them Sonelgaz, the national electricity provider, and private partners such as SAIEG, are beginning to deploy charging networks, with a national programme targeting 30,000 charging stations by 2030. SAIEG's export of 433 charging stations to Italy and Libya in 2024 further attests to the industrial maturity supporting this transition.

The management of charging infrastructure relies on internationally recognised technical standards. The ISO 15118 standard defines the vehicle-to-grid communication interface and constitutes the foundation of advanced features such as Plug & Charge and demand response. The Open Charge Point Protocol (OCPP), of which version 1.6 is widely adopted by equipment manufacturers, ensures communication between charging points and management systems. Mastery of both standards is indispensable to the design of a modern, interoperable charging management solution.

2. Problem Statement

Despite this dynamic, the deployment of EV charging infrastructure in Algeria faces concrete operational obstacles that hinder its development and compromise long-term viability:

- The absence of a mechanism for locating available charging stations and navigating to them constitutes a major barrier to EV adoption. The inability to consult the real-time availability of a charging point exposes users to unnecessary journeys toward occupied

or out-of-service equipment.

- Network management suffers from the absence of centralised supervision. Without a unified dashboard, it is impossible to obtain a fleet-wide view of stations, monitor ongoing sessions, analyse usage indicators, or generate reliable financial reports. This gap is compounded by the inability to control equipment remotely: every start, stop, configuration, or reset operation requires physical on-site intervention, incurring significant operational cost and delay.
- The absence of an integrated billing and payment system makes it difficult to collect revenue and track the financial activity of charging operations, compromising the economic viability of the network.
- Equipment maintenance is hampered by the lack of a structured incident management system. Fault reporting is informal and untraceable, intervention tracking is non-existent, and field maintenance agents have no digital tool to consult intervention details, update status, or document outcomes.
- The local market lacks a software solution built on internationally recognised open standards. This absence compromises interoperability with existing equipment and prevents exploitation of the advanced features defined by the ISO 15118 standard. Furthermore, the choice between different versions of the OCPP protocol requires rigorous comparative analysis to determine the version best suited to the Algerian context.

Taken together — the absence of station localisation, centralised supervision, remote management, integrated billing, structured maintenance tracking, and alignment with international standards — these gaps constitute the central problematic that this work seeks to address.

3. Objectives

The primary objective of this thesis is to design and develop **EV Charge Manager**, an integrated multi-tenant web platform accompanied by a complementary mobile application, adapted to the operational, technical, and regulatory constraints of the Algerian context. The solution aims to deliver a complete, coherent system covering the full range of identified needs, from station localisation to maintenance intervention tracking, through network supervision, session management, and billing.

In response to the identified problems, the following secondary objectives are defined:

- Implement a station localisation mechanism with real-time availability consultation, and integrate a GPS navigation system to guide the user to the chosen station.
- Develop a charging session management module enabling sessions to be initiated, monitored in real time, and reviewed through consumption history with push notification support.
- Design a centralised dashboard providing a complete overview of the charging network:

station status, active sessions, usage indicators, and analytical and financial reports.

- Implement remote station management via the OCPP 1.6 protocol, enabling start, stop, configuration, and reset operations without physical on-site intervention.
- Integrate an automated billing module ensuring invoice generation, per-session revenue tracking, and financial report production.
- Develop a maintenance ticket management system allowing incidents to be declared, categorised, assigned to technicians, and tracked through to resolution, with a mobile interface dedicated to field interventions.
- Conduct a comparative study of OCPP 1.6 and OCPP 2.0 and an analysis of the ISO 15118 standard, in order to justify the technical choices retained and position the platform within the international charging standards landscape.

4. Thesis Organisation

The work is organised as follows:

- **Chapter 1** constitutes the first part of the state of the art. It examines the ISO 15118 standard, which defines the vehicle-to-grid communication interface and underpins advanced charging features such as Plug & Charge and demand response.
- **Chapter 2** completes the state of the art with a comparative analysis of OCPP 1.6 and OCPP 2.0, justifies the protocol choice for this project in the Algerian context, and surveys existing EV charging management platforms.
- **Chapter 3** presents the requirements analysis for EV Charge Manager, identifying the system actors, specifying functional and non-functional requirements, and establishing the use cases and technical constraints that frame the design.
- **Chapter 4** details the architectural and design decisions of the platform, covering the three-tier architecture, multi-tenant isolation model, data model, access control structure, and authentication flow.
- **Chapter 5** presents the concrete implementation of all functional modules and demonstrates the operational behaviour of the deployed platform.
- **Chapter 6** presents the EVCharge Android mobile application, the driver-facing companion to the operator dashboard. It covers the technology stack, application architecture, shared backend integration, interactive station map, OpenRouteService navigation, real-time WebSocket updates, Firebase Cloud Messaging notifications, and session management from the driver perspective.

Declaration on the use of AI tools. During the realization of this work, AI-assisted tools specifically Claude AI (Anthropic) were used to support documentation drafting and code debugging. All content has been reviewed, corrected, and contextualised by the authors, who take full re-

sponsibility for the accuracy, originality, and integrity of this report.

State of the Art

Chapter 1

ISO 15118 Standard : Definition and Relevance to Smart EV Charging

This chapter examines the ISO 15118 international standard, which governs the communication interface between electric vehicles and charging infrastructure. We analyse its definition, its technical architecture, and its relevance to the design of advanced charging management systems, covering charging modes, dynamic parameter exchange, demand response, and security mechanisms. A reference proof-of-concept implementation is presented to illustrate the standard's practical feasibility [1].

1.1 What is ISO 15118?

ISO 15118 is an international standard that defines the communication interface between an Electric Vehicle (EV) and Electric Vehicle Supply Equipment (EVSE)—commonly known as a charging station [2]. It is officially titled “*Road Vehicles—Vehicle to Grid Communication Interface*” and is developed by the International Organization for Standardization (ISO).

The standard goes far beyond simple plug-and-charge detection. It establishes a full bidirectional digital communication protocol referred to as Vehicle-to-Grid (V2G) communication enabling EVs and charging infrastructure to exchange rich data in real time. This includes authentication, charge parameter negotiation, energy management, and support for grid services. The potential of V2G for grid stabilisation has been extensively studied: Yilmaz and Krein [3] demonstrated significant benefits in grid stability and utility revenue from managed EV charging.

ISO 15118 is structured across multiple parts (Parts 1–20), covering physical layer requirements, network and transport layers, application layer messaging, and security. The version most relevant to practical implementations is **ISO 15118-2**, which defines the application-layer message set used during a charging session [2].

1.2 Why ISO 15118 Matters for Smart EV Charging

Before ISO 15118, EV charging relied on the IEC 61851-1 standard [4], which uses simple low-level control pilot signaling. This approach only communicates two pieces of information: whether a vehicle is connected, and what maximum current the charger can supply. This severely limits smart charging capabilities.

ISO 15118 addresses these limitations by enabling:

- **Plug & Charge (PnC):** The EV is automatically identified and authorized using digital certificates, eliminating the need for RFID cards or apps.
- **Bidirectional Communication:** Both the EV and EVSE can send and receive detailed parameters such as State of Charge (SoC), target current/voltage, and power limits.
- **Demand Response:** The charging station can dynamically adjust power delivery based on grid conditions or renewable energy availability.
- **Vehicle-to-Grid (V2G):** In supported configurations, energy stored in the EV battery can be fed back to the grid, turning EVs into distributed storage assets [5].
- **Security:** Transport Layer Security (TLS)-based encrypted communication and mutual certificate authentication protect the session end-to-end.

1.3 Reference Implementation

A proof-of-concept study by Santos et al. [1] demonstrated the feasibility of building a fully ISO 15118-compliant charging system using the open-source RISE V2G Java library [6]. Rather than relying on simple current-signalling through IEC 61851-1 [4], their system implements full V2G digital communication, enabling intelligent charge management and demand response capabilities. The following sections describe the architecture and experimental results of this reference implementation, which serves as a key element of the state of the art for this project.

1.4 System Architecture of the Reference Implementation

Santos et al.'s system is built around two microcomputer modules: one representing the EVSE side (**EVSEpi**) and one representing the EV side (**EVpi**), each running on a Raspberry Pi 3 Model B. Communication between the SECC and EVCC is established over a Wi-Fi wireless network, leveraging the open-source **RISE V2G** Java library [6] as an ISO 15118-2 implementation as the communication engine.

Each module contains four sub-components:

- **Communication Controller (SECC/EVCC):** Handles the ISO 15118 V2G message exchange, extended to support dynamic parameter updates every 5 seconds overcoming the static-parameter limitation of the base RISE V2G library.

- **Parameter File:** Acts as a shared interface between the communication controller and the management code. It stores all charging parameters (current, voltage, SoC, flags) in a format compatible with the RISE V2G library.
- **Management Code:** A Java-based finite state machine that controls the charging process, updates parameters, and communicates with the HMI and IoT platform.
- **Human-Machine Interface (HMI):** Allows the EV user and Charge Point Operator (CPO) to configure parameters and monitor the charging session in real time.

1.5 DC Charging Mode and CC-CV Strategy

The reference implementation operates in DC power transfer mode, chosen because DC messaging carries a richer parameter set than AC, allowing finer charge management. The EV module emulates a virtual Li-ion battery using the industry-standard **Constant-Current / Constant-Voltage (CC-CV)** charging strategy [1]:

- **CC Phase (SoC 0–79 %):** The EVCC requests maximum current while voltage rises linearly.
- **CV Phase (SoC 80–100 %):** Voltage is held at its maximum while current steps down in decreasing increments until the battery is full.

1.6 Dynamic Parameter Exchange

A key enhancement reported by Santos et al. [1] is the ability to exchange dynamic parameters during an active charging session. Their system updates SoC, target current, target voltage, and remaining charge time **every 5 seconds** via the CurrentDemand V2G message loop. This enables the EVSE to respond in real time to changes in available power critical for renewable energy integration.

1.7 Demand Response with Photovoltaic Integration

One of the primary applications demonstrated in the Santos et al. implementation is demand response driven by photovoltaic (PV) generation. When PV output fluctuates, the EVSEpi management code adjusts the EVSE current and voltage limits accordingly. These updated limits are communicated to the EV via the CurrentDemandRes message, causing the vehicle to adapt its charge rate in near real time.

Experimental validation using a high-resolution PV dataset confirmed that a 5-second communication interval between SECC and EVCC produces a mismatch of only **0.38 %** between generation power and charging power an excellent balance between responsiveness and network efficiency (0.2 packets/second) [1].

1.8 IoT Integration and Monitoring

In the Santos et al. reference system, charging data is forwarded to an Emoncms IoT platform [7] every minute by the EVSEpi module. This allows the Charge Point Operator to monitor current, voltage, SoC, energy consumed (kWh), and charging cost over time via live dashboards demonstrating full operational visibility of ISO 15118 session data.

1.9 Security

ISO 15118 defines certificate-based authentication as the preferred payment and identity method through its Plug & Charge (PnC) flow [2]. Compared to External Identification Means (EIM) such as RFID cards, certificate-based PnC offers stronger security guarantees: the vehicle is automatically identified and authorised without physical interaction, relying on Transport Layer Security (TLS) and mutual certificate validation. The RISE V2G library provides a ready implementation of this mechanism [6], as demonstrated in the Santos et al. reference system.

1.10 Summary

Table 1.1: ISO 15118 Reference Implementation , Key Technical Parameters (Santos et al., 2024 [1])

Aspect	Detail
Standard	ISO 15118-2 (Application Layer — DC Conductive Charging) [2]
Library	RISE V2G (open-source Java implementation) [6]
Hardware	Raspberry Pi 3 Model B (EVSEpi + EVpi)
Communication	Wi-Fi (replacing standard powerline; same ISO 15118 messages)
Transfer Mode	DC (richer parameter set for smart charging)
Auth Method	Certificate-based (Plug & Charge)
Update Rate	5-second charging loops (CC-CV dynamic updates)
Demand Response	PV-integrated; 0.38 % power mismatch at 5 s interval
Monitoring	Emoncms IoT platform (1-minute telemetry) [7]

1.11 Conclusion

This chapter has examined the ISO 15118 standard and its practical application as demonstrated by Santos et al. [1]. The standard’s bidirectional V2G communication model, CC-CV charging

strategy, 5-second dynamic parameter exchange, and certificate-based Plug & Charge authentication collectively establish a technically rigorous foundation for smart charging management. The experimental validation a power-tracking mismatch of only 0.38 % under real photovoltaic conditions confirms the standard's suitability for demand-response scenarios and the feasibility of open-source ISO 15118 implementations.

Chapter 2 completes the state of the art by comparing the OCPP 1.6 and OCPP 2.0 protocol versions, justifying the protocol selection adopted for EV Charge Manager in the Algerian context, and surveying existing EV charging management platforms.

Chapter 2

OCPP 1.6 vs. OCPP 2.0 : A Comprehensive Comparison

The Open Charge Point Protocol (OCPP) is the backbone of interoperability in the electric vehicle (EV) charging industry, enabling seamless communication between charging stations and central management systems (CSMS) [8, 9]. While OCPP 1.6 laid the foundation for modern EV charging networks, OCPP 2.0 represents a significant leap forward, introducing advanced features to address the growing complexity of EV infrastructure.

In this chapter we explore the key differences between OCPP 1.6 and OCPP 2.0, highlighting how the latter supports scalability, advanced energy management, and enhanced security.

2.1 Device Management and Structure

OCPP 1.6:

- OCPP 1.6 defines connectors as independent charging outlets, each treated as a standalone unit.
- There is no explicit concept of Electric Vehicle Supply Equipment (EVSE), limiting its ability to represent complex configurations like multi-connector stations.

OCPP 2.0:

- OCPP 2.0 introduces a hierarchical device model, defining EVSE as a logical unit capable of managing one charging session at a time, even if multiple connectors are available.
- This formalized structure improves scalability and enables operators to manage advanced setups like multi-connector EVSEs and satellite systems efficiently [9].

2.2 Smart Charging Capabilities

OCPP 1.6:

- Smart charging is limited to predefined charging profiles [8]:
 - *TxProfile*: Specific to a single charging session.
 - *TxDefaultProfile*: Applies to all transactions on a connector.
 - *ChargingStationMaxProfile*: Sets an upper limit on the station’s total power consumption.
- Charging profiles in OCPP 1.6 are static and cannot adapt dynamically to real-time conditions, making it less flexible for complex energy management scenarios.

OCPP 2.0:

- Smart charging in OCPP 2.0 is more dynamic, allowing real-time updates to charging profiles based on factors such as grid conditions, energy prices, or vehicle schedules [9].
- *Recurring Profiles*: Enables daily or weekly schedules for predictable charging patterns.
- Supports integration with ISO 15118 for Plug & Charge and Vehicle-to-Grid (V2G) functionalities, providing bidirectional energy flow and automatic session initiation.

2.3 Message Handling and Communication

OCPP 1.6:

- Uses separate messages for related actions, such as `StartTransaction`, `StopTransaction`, and `MeterValues` [8].
- Message structures are simpler but less efficient for large-scale operations.

OCPP 2.0:

- Consolidates transaction-related messages into the unified `TransactionEvent` message, reducing redundancy and simplifying communication [9].
- Introduces new messages like `GetCompositeSchedule` for aggregating and visualising energy needs across a charging station.
- Supports WebSocket compression for more efficient communication, particularly useful for high-traffic networks.

2.4 Enhanced Security

OCPP 1.6:

- Basic security measures include Transport Layer Security (TLS) for encrypted communication [8].
- Authentication is limited to username-password combinations, which can be vulnerable if not properly configured.

OCPP 2.0:

- Implements robust security profiles [9]:
 - *Security Profile 1*: Basic unencrypted communication (not recommended).
 - *Security Profile 2*: Encrypted communication using TLS.
 - *Security Profile 3*: End-to-end encryption and mutual authentication using X.509 certificates.
- Introduces certificate management features for secure firmware updates and authentication, ensuring compliance with modern cybersecurity standards.

2.5 Firmware Management

OCPP 1.6:

- Supports remote firmware updates but lacks advanced monitoring and validation tools [8].

OCPP 2.0:

- Enhances firmware management with features like secure delivery via HTTPS and signature verification to prevent unauthorized installations [9].
- Includes detailed status updates through `FirmwareStatusNotification`, providing operators with real-time insights into the update process.

2.6 Backward Compatibility

OCPP 1.6:

- Compatible with earlier versions like OCPP 1.5, ensuring ease of transition for existing systems [8].

OCPP 2.0:

- Not backward compatible with OCPP 1.6 due to significant structural and functional changes [9]. Migration requires updating both CSMS and charger firmware.

2.7 Use Cases and Scalability

OCPP 1.6:

- Suitable for smaller, less complex charging networks with basic operational needs.
- Limited support for advanced use cases like V2G or complex energy optimization [10].

OCPP 2.0:

- Designed for large-scale, modern EV networks with advanced requirements like fleet management, renewable energy integration, and demand response programs [11].

- Scalable for multi-connector EVSEs and capable of managing sophisticated energy use cases.

2.8 Technical Comparison

Table 2.1: OCPP 1.6 vs. OCPP 2.0 — Technical Feature Comparison

Feature	OCPP 1.6 [8]	OCPP 2.0 [9]
Release	2015 (Ed. 2: 2019)	2020 (v2.0.1: 2022)
Adoption	Very high (worldwide)	Still growing
Smart charging	Static profiles	Dynamic real-time control
Security	Basic TLS	Advanced certificate-based
ISO 15118 integration	Limited	Native support
Device model	Simple (connector-centric)	Hierarchical (EVSE model)
Complexity	Low	High
Backward compatibility	OCPP 1.5	Not compatible with 1.6

Key takeaways from the comparison:

- OCPP 2.0 introduces advanced smart charging, stronger security, and better device management, but requires new hardware and a major migration effort.
- OCPP 1.6 remains the dominant protocol in deployed networks worldwide.
- Most charging stations and backend systems support OCPP 1.6 by default.
- OCPP 2.0 is still in early adoption stages [12].

2.9 Why OCPP 1.6 Was Chosen for This Project

2.9.1 Limited EV Infrastructure in Algeria

Algeria is currently in an early phase of electric vehicle deployment, characterized by limited charging infrastructure and the absence of an established smart grid or V2G ecosystem. The advanced functionalities introduced by OCPP 2.0 including dynamic smart charging, bidirectional energy management, and native ISO 15118 integration are therefore not operationally required in the present Algerian context.

2.9.2 Hardware Availability

In developing markets such as Algeria, the majority of commercially available low-cost charging hardware supports only OCPP 1.6. OCPP 2.0-certified equipment remains scarce and significantly more expensive, making OCPP 1.6 the economically viable and practically accessible choice for deployment [10].

2.9.3 Simplicity for Academic Prototype

OCPP 1.6 offers considerably greater simplicity and benefits from an extensive ecosystem of documentation, tutorials, and open-source tooling. Numerous reference implementations and protocol simulators are available for OCPP 1.6, facilitating rapid prototyping and academic experimentation within the scope of this project.

2.9.4 Network Constraints

OCPP 1.6 imposes significantly lower communication overhead than OCPP 2.0, making it more resilient in environments characterized by limited or intermittent network connectivity. Given Algeria's uneven internet infrastructure particularly outside major urban centers OCPP 1.6 provides a more reliable communication layer for deployed charging stations [11].

2.9.5 Industry Compatibility

OCPP 1.6 remains the dominant protocol in deployed charging networks worldwide. Its widespread adoption ensures interoperability across a broad range of vendor hardware and backend systems a critical factor when integrating with existing infrastructure in developing markets.

2.10 Overview of Existing EV Charging Management Platforms

Before designing our platform, we examined several established EV charging ecosystems to understand industry practices and identify gaps relevant to the Algerian context. The market generally structures solutions around two layers: a *driver-facing application* for station discovery and payment, and an *operator-facing backend* for fleet management, billing, and analytics. The following platforms are representative of this separation.

2.10.1 PlugShare and Driivz

PlugShare is a widely-used driver application that offers real-time station availability, trip planning, and community-sourced reviews across public and private charging networks. On the operator side, Driivz provides a Charging Station Management System (CSMS) built on OCPP, delivering energy optimization, dynamic load balancing, and white-label driver applications. The combination targets large-scale network operators seeking enterprise-grade monitoring and revenue management.

2.10.2 ChargePoint and ChargeLab

ChargePoint operates one of the largest charging networks globally, offering drivers a mobile application with access to over 140,000 stations, session reservations, and real-time status updates. ChargeLab complements this ecosystem with an open OCPP-based CSMS designed for hardware-agnostic deployments. It provides operators with station monitoring, configurable billing, and scalable backend infrastructure suitable for both small fleets and large public networks.

2.10.3 EV Connect and Tap Electric

EV Connect targets site hosts and fleet operators with a driver application providing access to a broad charging network alongside advanced reservation features. Tap Electric offers a lightweight, cost-free administration tool aimed at smaller operators, enabling tariff configuration, payment processing, and access control management without the overhead of enterprise platforms.

2.10.4 Positioning of EV Charge DZ

These platforms share a common limitation in the Algerian context: they are designed for established markets with mature payment infrastructure, reliable connectivity, and widespread EV adoption. None of them natively supports Algerian payment methods such as CIB or Baridi-Mob, multi-tenant operator isolation aligned with the Algerian regulatory environment, or the Arabic-French bilingual operational context.

EV Charge DZ addresses these gaps by combining OCPP 1.6 communication with a purpose-built multi-tenant administration platform that integrates local payment methods, fine-grained role-based access control, and a resilient offline fallback mechanism all tailored to the operational realities of the Algerian EV charging market.

Table 2.2: Comparison of Existing Platforms vs. EV Charge DZ

Feature	PlugShare / Driivz	ChargePoint / Charge-Lab	EV Connect / Tap Electric
Protocol	OCPP 1.6 / 2.0	OCPP 1.6 / 2.0	OCPP 1.6
Multi-tenant	Partial	Yes	Limited
Local payments	No	No	No
2FA (OTP)	No	No	No
Role granularity	Basic	Moderate	Basic
Offline fallback	No	No	No
Target market	Global	Global	Global

2.11 Conclusion

In this project, OCPP 1.6 was selected instead of OCPP 2.0 due to its widespread adoption, lower implementation complexity, and better compatibility with existing charging infrastructure. While OCPP 2.0 introduces advanced features such as enhanced security and dynamic smart charging, these functionalities are not required in the current Algerian context where EV deployment is still limited. OCPP 1.6 is supported by most available charging stations and offers sufficient capabilities for remote monitoring, transaction management, and basic smart charging. Therefore, OCPP 1.6 represents a practical and cost-effective choice for the proposed system [8].

Chapter 3 translates the insights from this state of the art into a concrete requirements analysis for EV Charge Manager, identifying system actors, functional and non-functional requirements, and the technical constraints that guided the design.

Design and Implementation

Chapter 3

Requirements Analysis and Specification

This chapter presents the requirements analysis for the EV Charge Manager platform. We identify the system actors, specify the functional and non-functional requirements, present the use case diagram, and outline the technical constraints that shaped the architectural decisions described in the following chapter.

3.1 Identification of System Actors

Three principal actors interact with the EV Charge Manager platform:

- **Super Administrator:** Manages the entire platform across all tenants. This actor has unrestricted access to all modules: stations, sessions, billing, maintenance tickets, user management, and push notifications.
- **Tenant Administrator:** Manages one operator’s charging network. Access is restricted to the data belonging to their tenant. This actor can supervise stations, sessions, billing, tickets, and push notifications within their scope.
- **Technician:** A field maintenance agent who accesses only the maintenance tickets assigned to them. This actor has no visibility into other modules or unrelated tickets.

An implicit external actor is the OCPP-compatible charging station hardware, which communicates with the backend system via the WebSocket-based OCPP 1.6 protocol [8].

3.2 Functional Requirements

The following functional requirements were identified through analysis of the operational needs of EV charging network operators in Algeria:

- **RF01 — Authentication:** The system shall allow users to authenticate via email and password, with mandatory OTP two-factor verification sent to the registered email address.

- **RF02 — Multi-Tenant Isolation:** The system shall enforce complete data isolation between tenants; each operator shall access only their own stations, sessions, tickets, billing records, alerts, notifications, and users.
- **RF03 — User Management:** Super administrators shall create, modify, suspend, and reactivate platform users, assigning them a role and privilege template.
- **RF04 — Operational Dashboard:** The system shall provide a real-time dashboard displaying seven KPIs (active stations, offline stations, under-maintenance stations, active sessions, energy delivered, revenue, and network uptime) with period filtering and drill-down panels.
- **RF05 — Station Management:** Operators shall manage the full lifecycle of charging stations, including creation with GPS coordinates, connector configuration, tariff setting, status control, and CSV export.
- **RF06 — Session Monitoring:** The system shall log all charging sessions with complete metadata (station, connector, user, duration, energy, cost, status) and allow filtering and CSV export.
- **RF07 — Billing:** The system shall support billing in Algerian Dinars (DZD) for three payment methods: RFID card, CIB bank card, and e-payment, with per-method performance analytics.
- **RF08 — Maintenance Tickets:** Operators shall create maintenance tickets with category, priority, SLA deadline, and technician assignment. The system shall escalate tickets automatically when their SLA deadline is breached.
- **RF09 — Technician Task Queue:** Technicians shall view and update only the tickets assigned to them, with full access to the activity log.
- **RF10 — Offline Degradation:** The frontend shall remain operational in read-only mode when the backend is unreachable, serving data cached in `localStorage`.

3.3 Non-Functional Requirements

- **RNF01 — Performance:** Dashboard KPIs must refresh within two seconds; REST API response time must not exceed 500 ms under normal operating load.
- **RNF02 — Security:** All API endpoints must be protected by JWT bearer token authentication [13]; passwords must be hashed using `bcrypt`; every login must complete OTP verification before a token is issued.
- **RNF03 — Availability:** The platform must degrade gracefully when the backend is unreachable, displaying a visible offline-mode banner and serving cached data without crashing.
- **RNF04 — Scalability:** The multi-tenant architecture must support additional operators

without code changes, relying on `tenant_id` scoping at the database level.

- **RNF05 — Maintainability:** The codebase must use TypeScript strict typing throughout; the API must expose a consistent RESTful interface with clear endpoint naming.
- **RNF06 — Portability:** The platform must run on any modern browser (Chrome, Firefox, Edge) without requiring native application installation.

3.4 Use Case Diagram

The use case diagrams below illustrate the interactions between the three system actors and the platform’s functional modules, presented separately per actor to clearly delineate access scope. The super administrator has unrestricted access across all modules (Figure 3.1). The tenant administrator’s scope is restricted to their operator’s data (Figure 3.2). The technician’s access is limited exclusively to their assigned maintenance tasks (Figure 3.3).

Note: A combined use case diagram grouping all three actors was initially produced but proved too large to be displayed clearly in the body of the report. The full version is provided in Appendix A for reference.

3.5 Technical Constraints

3.5.1 Protocol Constraint — OCPP 1.6

The platform must implement OCPP 1.6 for station communication [8]. As established in Chapter 2, OCPP 2.0.1 was evaluated but excluded due to limited hardware availability and network infrastructure maturity in Algeria.

3.5.2 Regulatory and Market Constraints

All billing must be expressed in Algerian Dinars (DZD). Payment integration must support the CIB interbank card network and local e-payment services, reflecting the payment landscape available to Algerian EV operators and end users.

3.5.3 Network Infrastructure Constraints

Given variable connectivity conditions in certain Algerian deployment areas, the frontend must retain functionality in partial offline scenarios. Operators must not lose access to operational data during temporary backend outages.

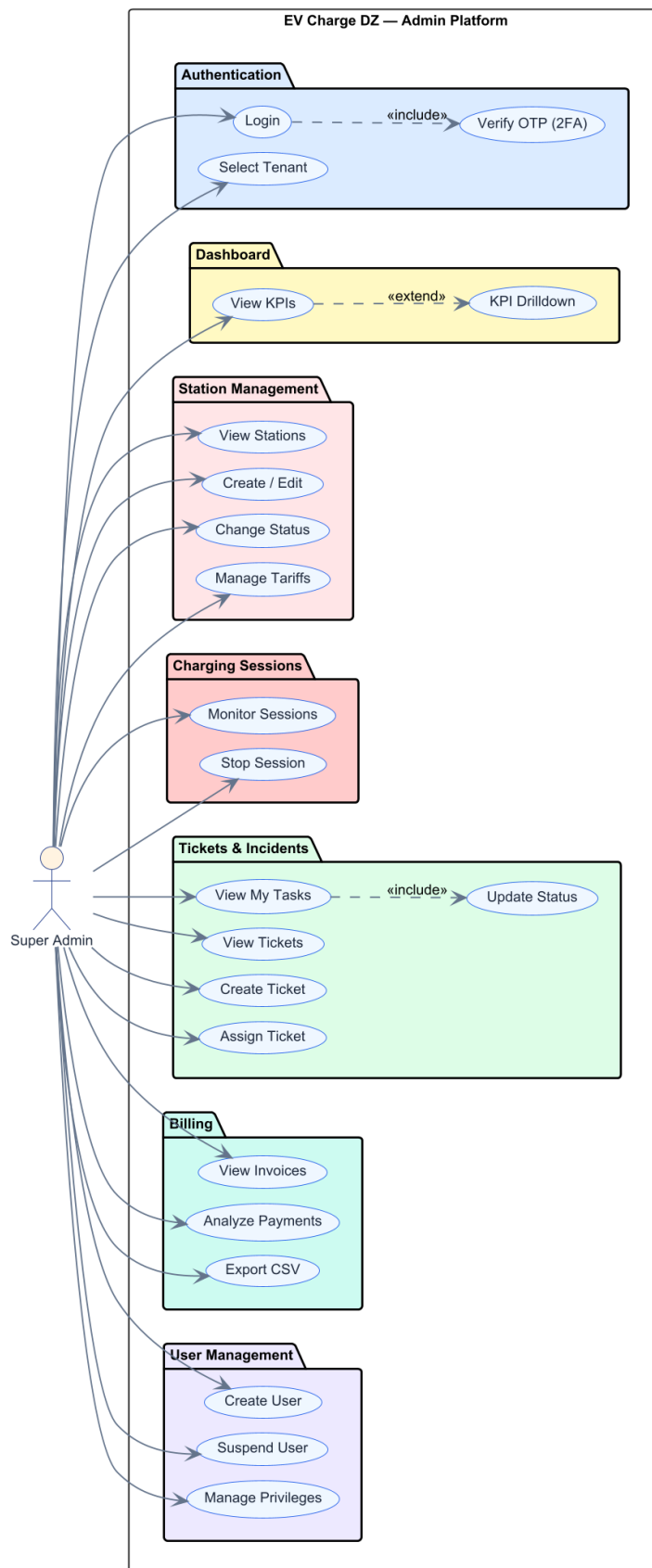


Figure 3.1: Use Case Diagram — Super Admin

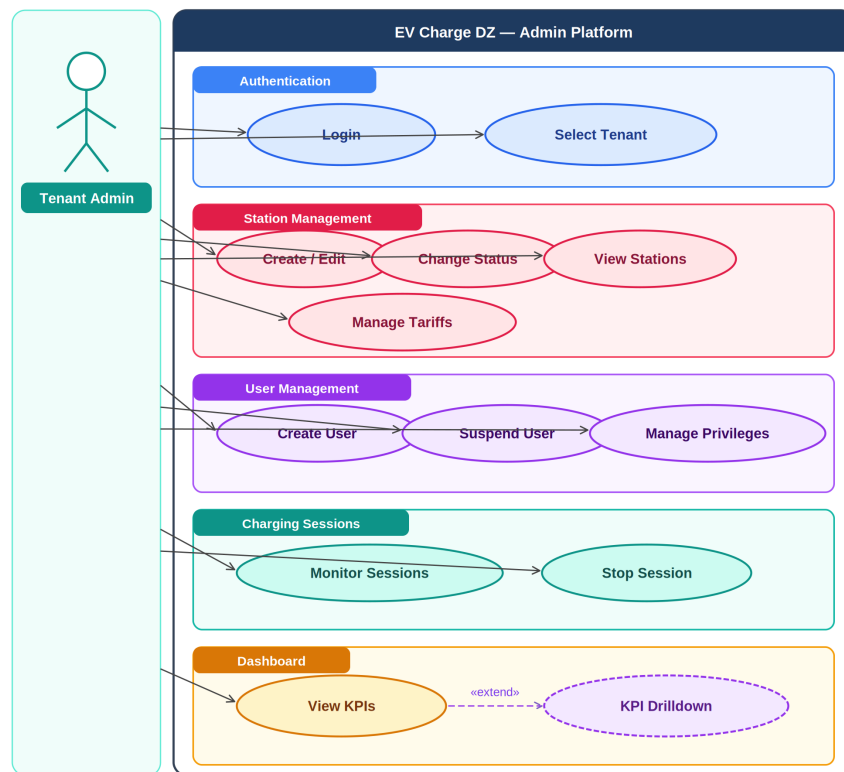


Figure 3.2: Use Case Diagram — Tenant Admin

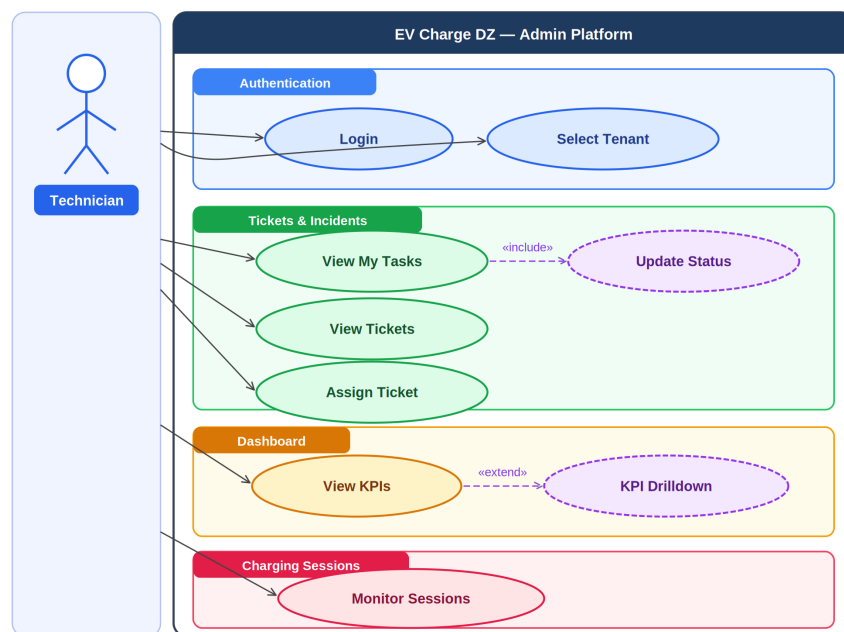


Figure 3.3: Use Case Diagram — Technician

3.6 Conclusion

This chapter has established the complete requirements baseline for the EV Charge Manager platform. Ten functional requirements and six non-functional requirements, combined with three categories of technical and regulatory constraints, define the scope and boundaries of the system. These requirements directly inform the architectural choices presented in Chapter 4 and are validated by the implementation demonstrated in Chapter 5.

Chapter 4

EV Charge Manager : Architecture and Design

This chapter presents the architectural and design decisions for EV Charge Manager, translating the requirements established in Chapter 3 into a concrete technical blueprint. We describe the general three-tier architecture, the multi-tenant isolation model, the data model, the user access control structure, the authentication flow, and the technology stack selected for the implementation.

4.1 General Architecture

The platform follows a standard three-tier architecture:

- **Presentation Tier:** A single-page application (SPA) built with React 18 and TypeScript [14, 15], bundled with Vite [16]. It runs entirely in the browser and communicates with the backend over HTTP/JSON.
- **Logic Tier:** A Node.js REST API built with Express.js [17], responsible for authentication, data validation, and business logic.
- **Data Tier:** A MySQL 8 relational database [18] storing all operational data across 13 tables: tenants, users, privileges, user privileges, mobile app users, stations, connectors, charging sessions, tickets, ticket activities, billing records, alerts, and push notifications.

The frontend communicates with the backend via a centralized API client module. All requests are authenticated using JSON Web Tokens (JWT) [13]. The application is designed to degrade gracefully: when the backend is unreachable, it falls back to locally persisted data (browser `localStorage`) and notifies the operator with a visible offline-mode banner.

Table 4.1: EV Charge Manager : Technology Stack

Layer	Technology	Role
Language	TypeScript 5	Type-safe JavaScript [15]
UI Framework	React 18	Component-based SPA [14]
Build Tool	Vite 6	Fast dev server and bundler [16]
UI Components	shadcn/ui + Radix UI	Accessible component primitives
Styling	Tailwind CSS	Utility-first CSS [19]
Charts	Recharts	SVG-based data visualisation [20]
Routing	React Router v7	Client-side navigation [21]
Map	Leaflet / Open-StreetMap	Interactive station map
Backend	Node.js + Express.js	REST API server [17]
Database	MySQL 8	Relational data storage [18]
Authentication	JWT + OTP (2FA)	Stateless token auth [13]
Charging Protocol	OCPP 1.6	Station communication [8]

4.2 Technology Stack

4.3 Multi-Tenant Architecture

The platform supports multiple independent operators referred to as *tenants* each with complete data isolation. Two tenants are currently configured:

- **Sonelgaz**: Algeria’s national electricity company and primary EV infrastructure operator.
- **SAEIG**: A private EV charging network operator.

Tenant isolation is enforced at the database level through a `tenant_id` foreign key on all operational entities (users, stations, charging sessions, tickets, billing records, alerts, and notifications). At the application level, a *TenantContext* React context propagates the active tenant across all components, which filter their data accordingly. Each tenant may also define custom branding (accent color, logo), providing a white-label experience while sharing common platform infrastructure.

4.4 Data Model

The class diagram in Figure 4.1 presents the database-oriented data model of EV Charge Manager. The schema comprises 13 tables organised across three concerns: dashboard staff management (users, privileges, user_privileges), mobile driver accounts

(`app_users`), and operational data (`stations`, `connectors`, `charging_sessions`, `tickets`, `ticket_activities`, `billing_records`, `alerts`, `notifications`). All operational entities carry a `tenant_id` foreign key, enforcing the multi-tenant isolation described in the previous section. Stations expose connectors for session initiation; completed sessions are aggregated into billing records. Tickets reference both a station and an assigned technician, and maintain a full activity log through `ticket_activities`. The `app_users` table stores mobile driver accounts alongside their FCM device token, which the `notifications` table uses to record and target push alerts dispatched from the operator dashboard.

Note: The complete class diagram including all enumeration types (`UserRole`, `StationStatus`, `SessionStatus`, etc.) is provided in Appendix B for reference.

4.5 User Roles and Access Control

The platform implements Role-Based Access Control (RBAC) with three distinct user roles:

Table 4.2: User Roles and Module Access

Role	Scope	Accessible Modules
super_admin	All tenants	Dashboard, Stations, Sessions, Billing, Tickets, Users, Notifications
tenant_admin	Own tenant only	Dashboard, Stations, Sessions, Billing, Tickets, Notifications
technician	Own tasks only	My Tasks (assigned maintenance tickets)

Beyond role assignment, the platform supports thirteen fine-grained privilege types organised into four profiles aligned with the actual roles of the system. Table 4.3 details the exact privilege set assigned to each profile. The *super_admin* profile grants unrestricted access across all 13 flags, covering every module of the platform and all tenants. The *tenant_admin* profile covers the full operational scope of a tenant — stations, sessions, billing, and tickets — but excludes user management and global settings, which remain reserved for the super administrator. The *finance_admin* profile is scoped to billing and reporting operations, with contextual read access to stations, sessions, and tickets; access to user management and station control is intentionally withheld. The *technician* profile is the most restricted, limited to the flags strictly required for on-site infrastructure maintenance, and explicitly omits billing and user data in compliance with GDPR data-minimisation requirements [22]. Routes in the frontend are guarded by a `RequireAuth` component that verifies both authentication and role before rendering any protected page.

Table 4.3 reveals a clear privilege hierarchy across the four role profiles. The *super_admin* holds all 13 flags, confirming its unrestricted cross-tenant authority over every module of the platform. The *tenant_admin* profile, with 10 flags, covers the full operational scope of

EV Charge DZ — Class Diagram (Database-Oriented)

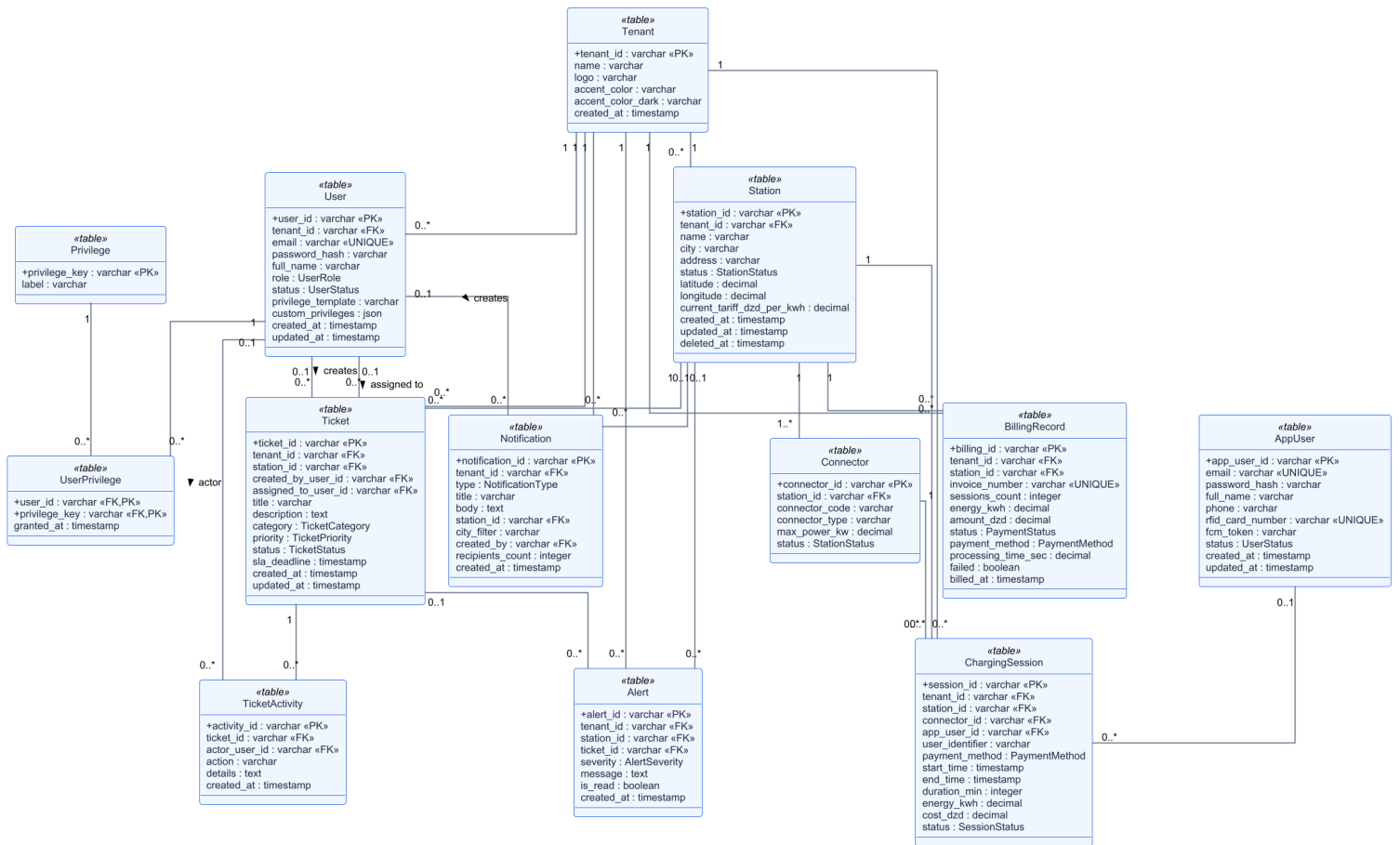


Figure 4.1: Class Diagram EV Charge DZ (Database-Oriented)

Table 4.3: Privilege Matrix — Assigned Flags per Role Profile

Privilege flag	super_admin	tenant_admin	technician	finance_admin
users_view	✓			✓
users_manage	✓			
stations_view	✓	✓	✓	✓
stations_manage	✓	✓	✓	
sessions_view	✓	✓	✓	✓
sessions_control	✓	✓	✓	
tickets_view	✓	✓	✓	✓
tickets_manage	✓	✓	✓	
billing_view	✓	✓		✓
billing_manage	✓	✓		✓
reports_view	✓	✓		✓
reports_export	✓	✓		✓
settings_manage	✓			
Total flags	13	10	6	8

a tenant while remaining isolated from global user management and platform settings. The *finance_admin* profile holds 8 flags focused exclusively on billing, reporting, and contextual read access, without any control over infrastructure or users. The *technician* profile is the most restricted at 6 flags, retaining only what is strictly necessary for on-site maintenance; access to billing and user data is intentionally excluded to comply with the GDPR data-minimisation principle.

4.6 Authentication Design

User authentication follows a two-factor flow:

1. The user submits their email, password, and tenant identifier on the login screen.
2. The backend validates the credentials and generates a One-Time Password (OTP) sent to the registered email address.
3. The user enters the OTP on a dedicated verification screen. On successful validation, the backend issues a signed JWT [13].
4. All subsequent API requests include the JWT as a Bearer token in the HTTP Authorization header. The backend validates the token signature and tenant claims on every protected endpoint.

The JWT is stored in the browser's `localStorage` and is cleared on logout. The `RequireAuth` component performs a client-side token presence check and role verification before rendering any dashboard route. The sequence diagram in Figure 4.2 illustrates the complete authentication and OTP verification flow for a super administrator.

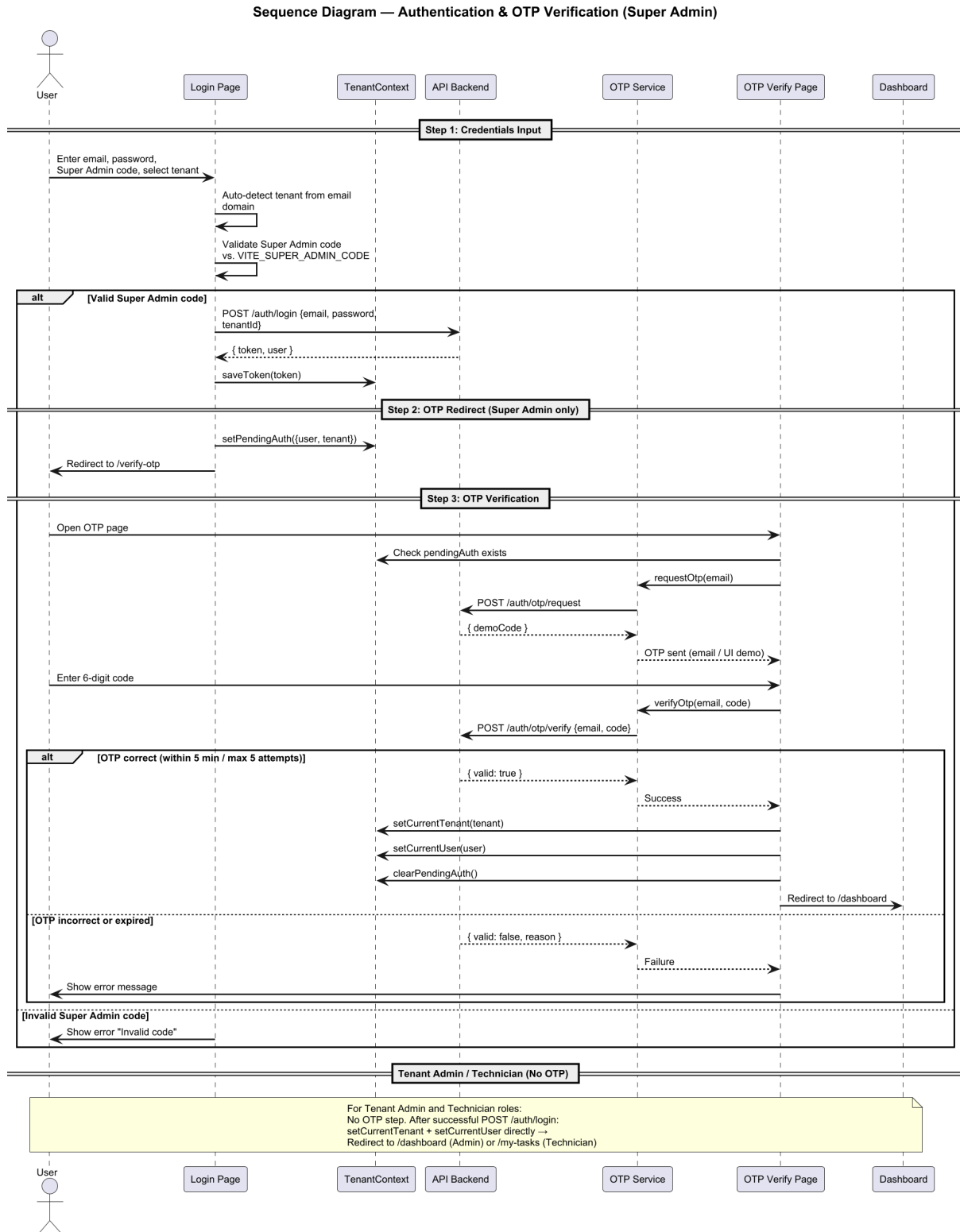


Figure 4.2: Sequence Diagram Authentication and OTP Verification Flow (Super Admin)

4.7 Conclusion

The architectural choices presented in this chapter three-tier separation, tenant-scoped database isolation, RBAC with thirteen fine-grained privileges, and two-factor JWT authentication directly address the functional and non-functional requirements identified in Chapter 3. Together they constitute the technical blueprint for the platform implementation detailed in Chapter 5.

Chapter 5

EV Charge Manager : Implementation and Testing

This chapter presents the concrete implementation of EV Charge Manager, module by module. Each section describes the operational interface and behaviour of a functional component, demonstrating how the design decisions of Chapter 4 translate into a working charging management platform. We also describe the development environment and conclude with a comprehensive platform summary.

5.1 Development Environment

The platform was developed on Windows 11 using Node.js 18 LTS for the backend runtime, MySQL 8 Community Edition as the database server, and the Vite development server for the React frontend. Visual Studio Code was used as the primary integrated development environment, with ESLint and Prettier extensions enforcing code style consistency across the codebase. Git was used for version control, with the project repository hosted on GitHub.

5.2 Authentication Interface

Access to EV Charge Manager is protected by a tiered authentication scheme adapted to each role's privilege level. Tenant admins and technicians authenticate with email and password and are directed immediately to their respective workspace. Super_admin accounts additionally require a static platform access code and a time-limited six-digit one-time password (OTP), forming a three-factor authentication chain that reflects the role's cross-tenant authority.

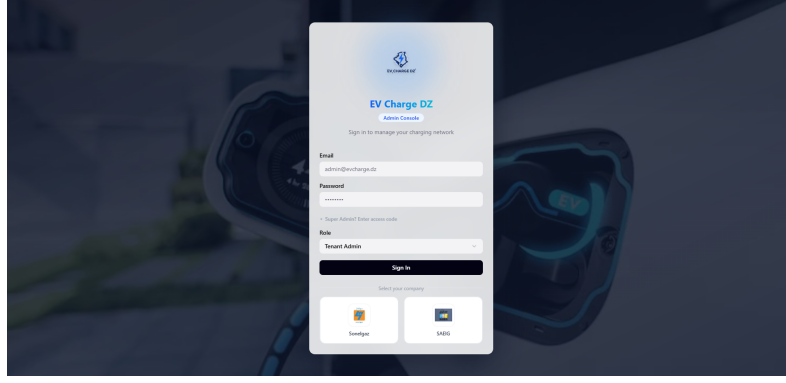


Figure 5.1: Login Page — Credential Entry and Tenant Selection

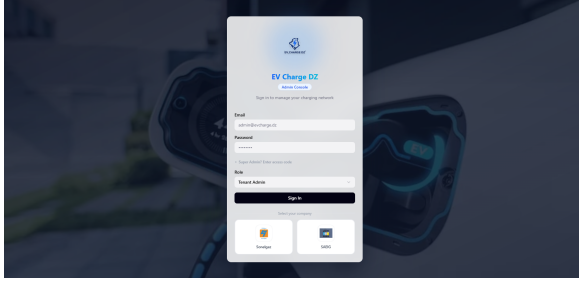


Figure 5.2: Login — Super Admin Access Code Field

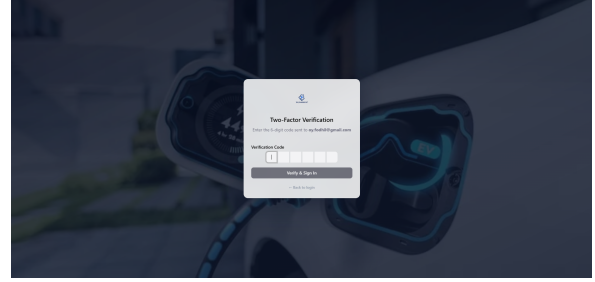


Figure 5.3: Two-Factor Verification — OTP Entry Step

5.3 Dashboard Module

The dashboard provides a real-time overview of the entire charging network through seven Key Performance Indicators (KPIs), displayed as interactive cards:

- **Active Stations:** Stations in *available* or *charging* status.
- **Offline Stations:** Stations in *fault* or *offline* status.
- **Under Maintenance:** Stations in *maintenance* mode.
- **Active Sessions:** Currently ongoing charging sessions.
- **Energy Delivered:** Total energy dispatched (kWh) over the selected period.
- **Revenue:** Cumulative billing amount (DZD) over the selected period.
- **Network Uptime:** Weighted average station availability rate (%).

Each KPI card displays a trend indicator comparing the current period to the previous one, and supports a clickable drilldown that opens a contextual side panel. The dashboard also includes:

- A 30-day revenue and energy trend chart (multi-line chart built with Recharts [20]).
- A connector-type distribution chart (donut chart showing CCS2 / CHAdeMO / Type 2 share).
- A 24-hour energy consumption curve (hourly resolution).
- A real-time alerts feed categorized by severity (info, warning, error).

A date filter (Today / 7 Days / 30 Days) is persisted in localStorage to maintain operator preferences across sessions.

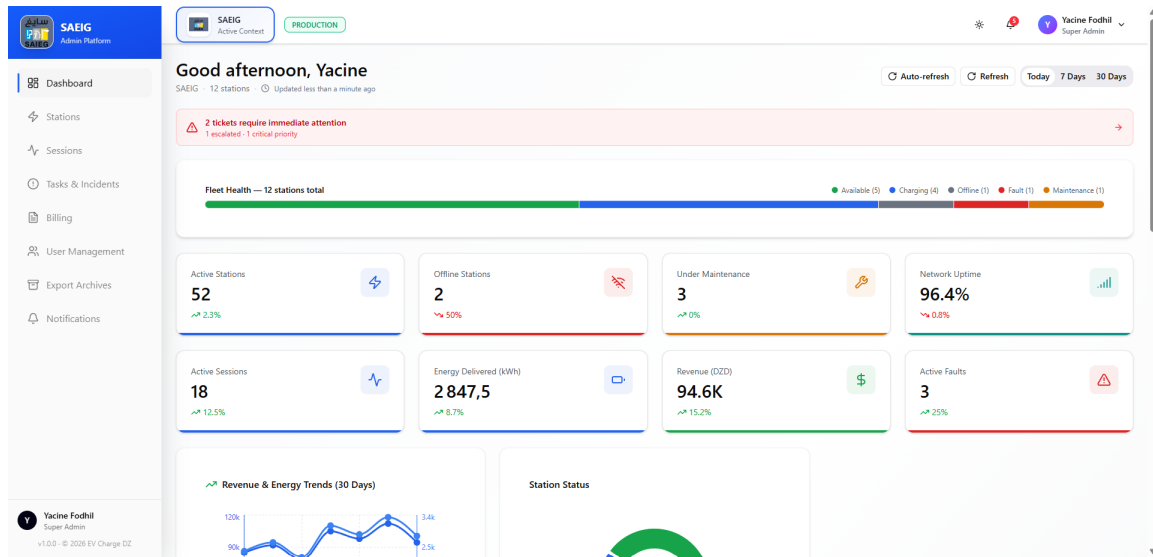


Figure 5.4: Dashboard Module Real-Time KPI Overview and Trend Charts

5.4 Station Management Module

The Stations module provides complete lifecycle management for charging infrastructure. It offers two complementary views:

- **Table View:** A searchable, filterable table with filters for status and city.
- **Map View:** An interactive Leaflet/OpenStreetMap map showing each station as a color-coded marker (by operational status). Clicking a marker reveals station details.

5.4.1 Connector Standards

Each station may host multiple connector types simultaneously. The platform supports the three main standards deployed in Algeria and internationally:

Table 5.1: Supported EV Connector Standards

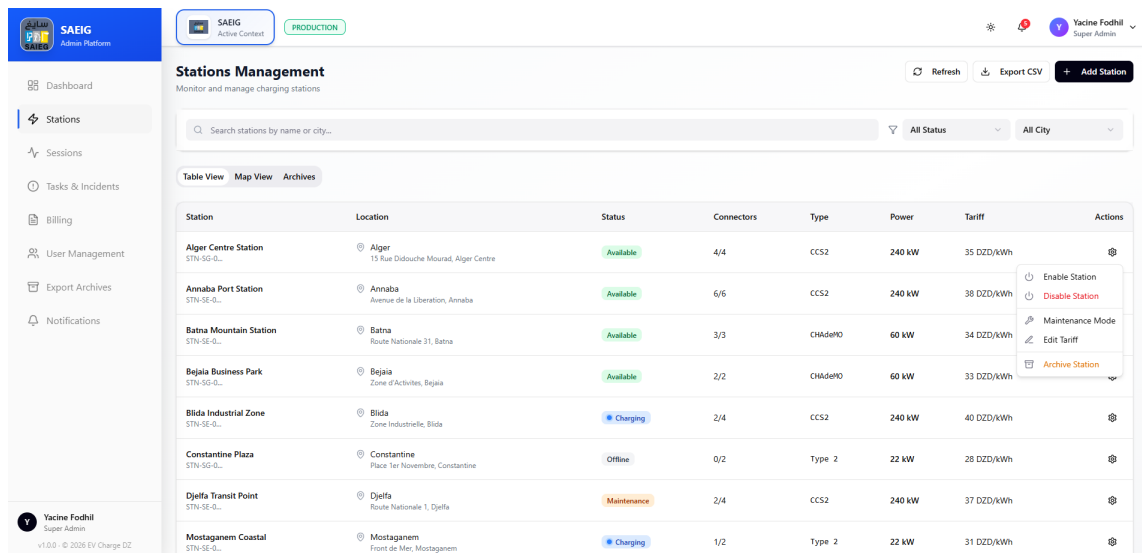
Type	Current	Typical Power	Use Case
CCS2	DC	Up to 350 kW	Fast and ultra-fast DC charging
CHAdMO	DC	Up to 400 kW	Japanese DC standard
Type 2	AC	3.7–22 kW	AC home and public slow charging

5.4.2 Station Status Values

Stations cycle through five operational states: *available* (idle, ready), *charging* (session active), *offline* (network lost), *fault* (hardware or software fault detected via OCPP [8]), and *maintenance* (under scheduled service).

5.4.3 Station Actions

Operators can: enable a station, disable it, toggle maintenance mode, update the tariff (DZD/kWh), and create a new station with GPS coordinates selected via an embedded location picker map. All status-changing actions require confirmation through a modal dialog and are logged in the system audit trail. A CSV export function allows operators to download the current filtered station list for external reporting.



The screenshot displays the 'Stations Management' interface. It includes a sidebar with navigation links (Dashboard, Stations, Sessions, Tasks & Incidents, Billing, User Management, Export Archives, Notifications) and a top header with user information (Yacine Fodhil, Super Admin). The main area shows a table of stations with columns: Station, Location, Status, Connectors, Type, Power, Tariff, and Actions. A search bar and filters for 'All Status' and 'All City' are present. A dropdown menu for the 'Actions' column is open, showing options: Enable Station, Disable Station, Maintenance Mode, Edit Tariff, and Archive Station.

Station	Location	Status	Connectors	Type	Power	Tariff	Actions
Alger Centre Station STN-SG-0...	Alger 15 Rue Didouche Mourad, Alger Centre	Available	4/4	CCS2	240 kW	35 DZD/kWh	⚙️
Annaba Port Station STN-SG-0...	Annaba Avenue de la Libération, Annaba	Available	6/6	CCS2	240 kW	38 DZD/kWh	⚙️
Batna Mountain Station STN-SG-0...	Batna Route Nationale 31, Batna	Available	3/3	CHAdeMO	60 kW	34 DZD/kWh	⚙️
Bejaia Business Park STN-SG-0...	Bejaia Zone d'Activités, Bejaia	Available	2/2	CHAdeMO	60 kW	33 DZD/kWh	⚙️
Blida Industrial Zone STN-SG-0...	Blida Zone Industrielle, Blida	Charging	2/4	CCS2	240 kW	40 DZD/kWh	⚙️
Constantine Plaza STN-SG-0...	Constantine Place 1er Novembre, Constantine	Offline	0/2	Type 2	22 kW	28 DZD/kWh	⚙️
Djelfa Transit Point STN-SG-0...	Djelfa Route Nationale 1, Djelfa	Maintenance	2/4	CCS2	240 kW	37 DZD/kWh	⚙️
Mostaganem Coastal STN-SG-0...	Mostaganem Front de Mer, Mostaganem	Charging	1/2	Type 2	22 kW	31 DZD/kWh	⚙️

Figure 5.5: Station Management — Table View with Status and City Filters

5.5 Session Monitoring Module

The Sessions module provides a chronological log of all charging sessions associated with the active tenant. Each session record contains: a unique session identifier, the source station name, the connector used, the user's identifier (RFID tag or account ID), session start time, duration (minutes), energy delivered (kWh), billing amount (DZD), and final status.

Session status values are: *active* (ongoing), *completed* (ended normally), *stopped* (manually interrupted by user or operator), or *error* (abnormal termination). Operators can filter sessions by status and station, and export results to CSV.

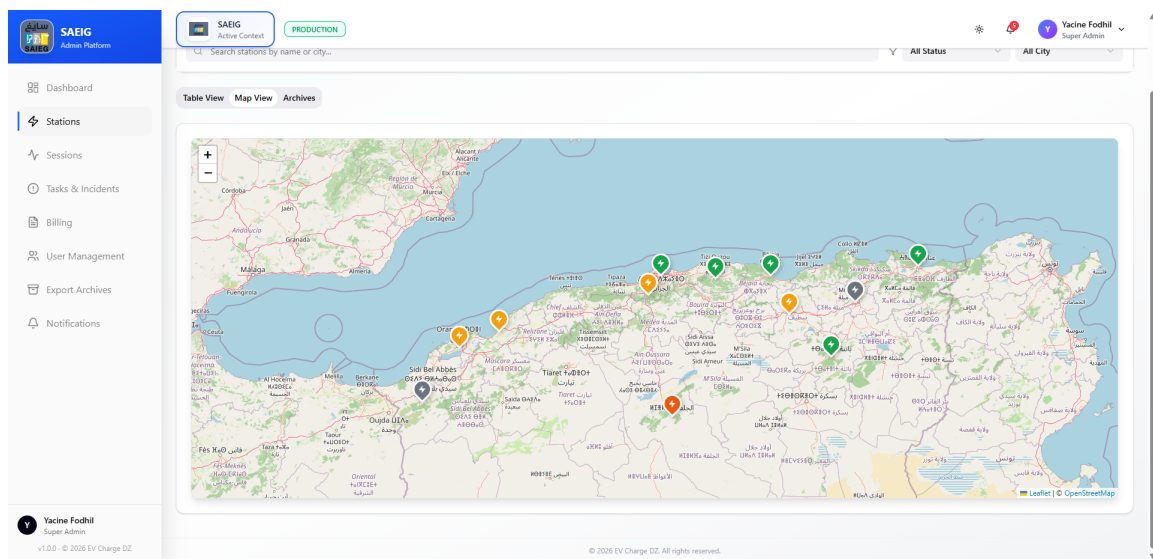


Figure 5.6: Station Management Interactive Map View (Leaflet / OpenStreetMap)

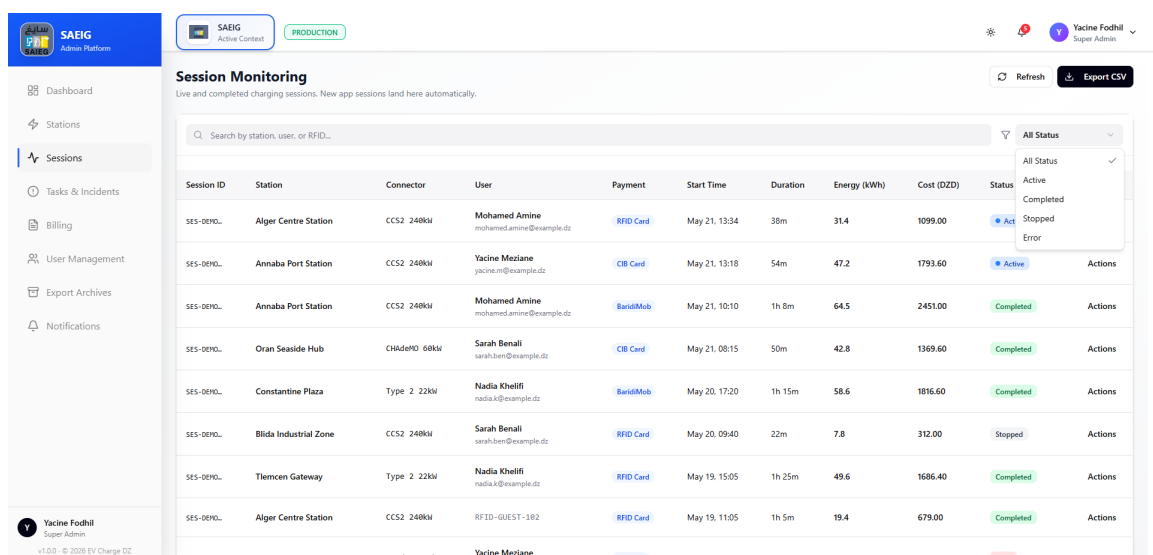


Figure 5.7: Session Monitoring Module Chronological Charging Session Log

5.6 Billing Module

The Billing module tracks all financial transactions generated by charging sessions. It supports three payment methods common in the Algerian market:

- **RFID Card:** The most widely deployed method; the customer authenticates using an RFID tag registered to their account.
- **CIB (Carte Interbancaire):** Electronic payment via the Algerian interbank bank card network.
- **E-Payment:** Mobile wallet or online payment gateway.

Each billing record includes an invoice number, associated station, number of sessions aggregated, total energy consumed (kWh), total amount (DZD), payment status (paid, unpaid, or refund), and the payment processing timestamp. The module provides per-method performance statistics, including transaction count, success rate, failed transaction count, average processing time (seconds), and revenue contribution percentage.

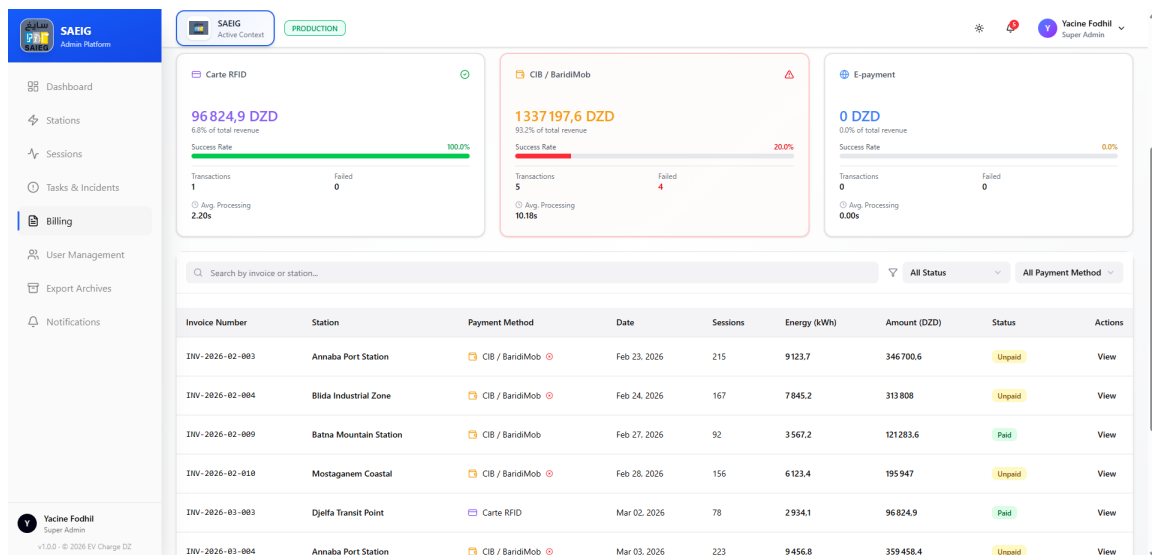


Figure 5.8: Billing Module Payment Method Statistics and Invoice Records

5.7 Maintenance Ticket System

The Ticket module implements a structured fault-reporting and maintenance-scheduling workflow. When a station issue is detected automatically via the alert system or manually by an operator a ticket is created with the following attributes:

- **Category:** One of six types power failure, screen issue, charger fault, system failure, network issue, or scheduled maintenance.
- **Priority:** Low, medium, high, or critical.

- **SLA Deadline:** A contractual resolution deadline tracked and displayed as a countdown; breached deadlines trigger automatic escalation.
- **Assignment:** The ticket is assigned to a specific technician.
- **Activity Log:** Every status change, comment, and action is recorded with a timestamp and actor identity, providing a complete audit trail.

The ticket lifecycle follows four states, as illustrated in Figure 5.9: *open* → *in_progress* → *resolved* (or *escalated* if the SLA deadline is breached without resolution).

Technicians access their work queue through the dedicated *My Tasks* page, which displays only tickets assigned to the currently logged-in user, allowing them to focus exclusively on their own workload without visibility into unrelated tickets.

5.8 User Management Module

The User Management module is accessible by default to *super_admin* accounts only. A *tenant_admin* account may also access it if a *super_admin* explicitly grants the *users_view* and *users_manage* privilege flags through the custom privilege template. This allows operators to delegate user administration within their own tenant scope without granting full cross-tenant authority. When creating a user, the administrator assigns: a role (*super_admin*, *tenant_admin*, or *technician*), a tenant affiliation, and a privilege template. As specified in Table 4.3 (Section 4.5), each template automatically assigns a predefined set of privilege flags that reflects the operator’s operational scope: *standard_admin* grants all thirteen flags (full platform access); *operations_admin* covers infrastructure and session control but excludes billing (9 flags); *finance_admin* provides read-only billing and reconciliation access (8 flags); and *technician_default* is scoped strictly to charge points and maintenance tickets (6 flags), omitting user data and billing in compliance with GDPR data-minimisation requirements. The *custom* template allows the administrator to manually select any combination of individual privileges.

User accounts can be suspended (blocking login without deletion) and reactivated, providing full lifecycle management of operator personnel.

5.9 Push Notifications Module

The Push Notifications module enables operators to broadcast real-time alerts to all registered mobile app users within a tenant. A compose panel allows the sender to specify a notification title, message body, an optional city filter (to target users in a specific geographic area), and for *super_admin* accounts a target tenant. Sent notifications are recorded in a history panel displaying the delivery count, timestamp, and notification type badge. The system supports five notification categories: *station_available*, *station_maintenance*, *tariff_change*, *new_station*, and

Activity Diagram — Maintenance Ticket Lifecycle

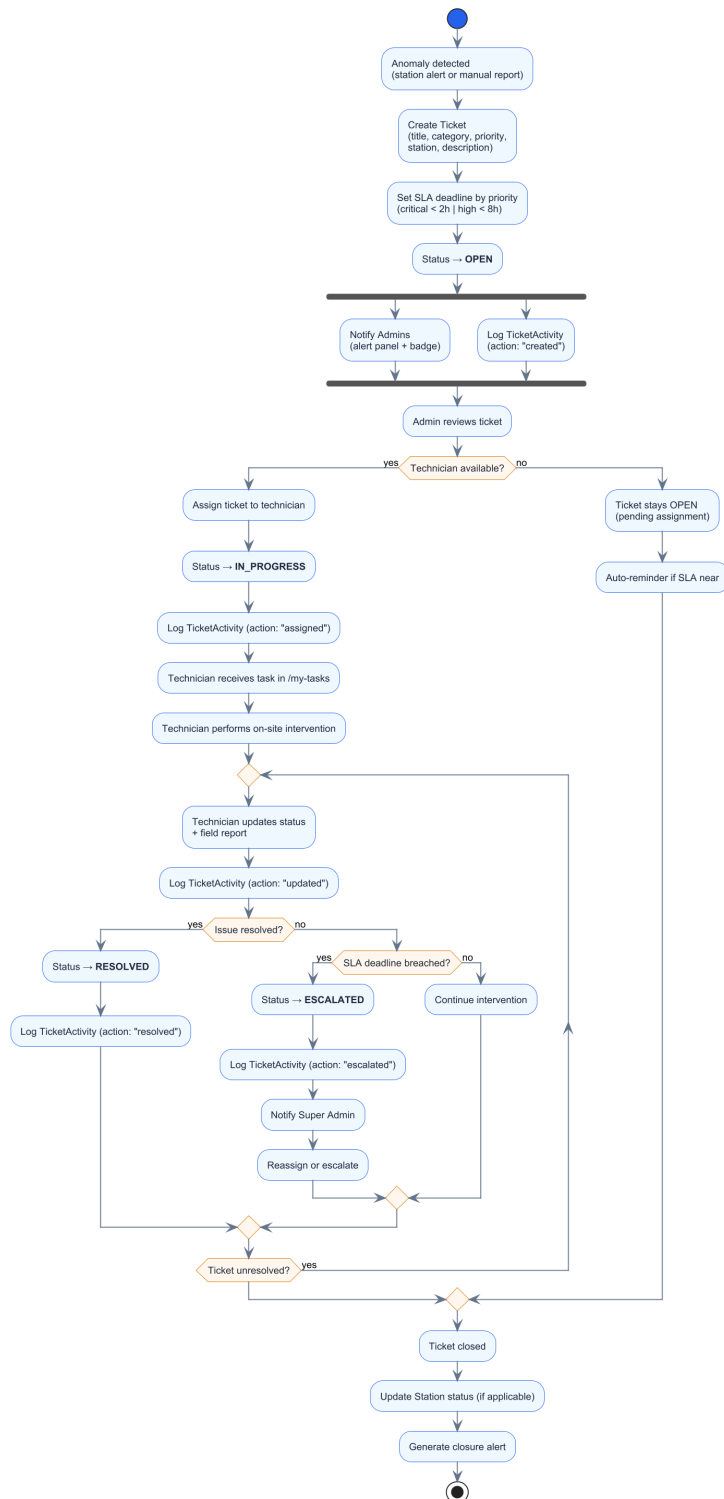


Figure 5.9: Activity Diagram Maintenance Ticket Lifecycle

The screenshot displays the SAEIG Maintenance Ticket Module. On the left is a sidebar with navigation links: Dashboard, Stations, Sessions, Tasks & Incidents (selected), Billing, User Management, Export Archives, and Notifications. The main header shows 'SAEIG Active Context' and 'PRODUCTION' status. The 'Tickets' section has a subtitle 'All incident reports and maintenance tasks across every tenant. Updates live as app users send reports.' Below this is a search bar and a table of tickets.

Ticket	Title	Station	Category	Priority	Status	SLA
4ec711c7-	Power / slow charging at Alger Centre Station	Alger Centre Station	Slow charging / power	High	Escalated	BREACHE May 31, 22
b342a3f1-	Screen screen freezing	Bida Industrial Zone	Screen issue	Medium	Open	—
ad7ee449-	Damaged equipment at Tizi Ouzou Central connector head install g9	Tizi Ouzou Central	Damaged equipment	Low	Escalated	BREACHE May 27, 17
9438f4c6-	System failure at Batna Mountain Station / cant charge / charging denied	Batna Mountain Station	System failure	Critical	Resolved	BREACHE May 23, 22
c744a3e2-	Power / slow charging at Bida Industrial Zone it is taking sooooo long	Bida Industrial Zone	Slow charging / power	High	Escalated	BREACHE May 22, 05
TXT-DEVL	Screen not responding Touch screen intermittently freezes during payment step.	Bejaia Business Park	Screen issue	Medium	Escalated	BREACHE May 22, 16
TXT-DEVL	Connector latch inspection CCS2 connector latch reported as stiff after repeated sessi...	Alger Centre Station	Damaged equipment	Low	Escalated	BREACHE May 19, 16
TXT-DEVL	Voltage anomaly detected Station reported voltage irregularities and connector fault ...	Bida Industrial Zone	Slow charging / power	Critical	Escalated	BREACHE May 21, 18
TXT-DEVL	Network latency on payment terminal	Annaba Port Station	Network / payment	High	Escalated	BREACHE

On the right, the 'New Ticket' form is open, titled 'Create an incident or maintenance task'. It includes fields for Tenant, Station, Title, Description, Category, Priority, Assign to, and SLA Deadline, along with 'Cancel' and 'Create Ticket' buttons.

Figure 5.10: Maintenance Ticket Module Ticket List and New Ticket Creation Form

The screenshot shows the 'My Tasks' section for a technician. The sidebar is the same as in Figure 5.10. The main header shows 'SAEIG Active Context' and 'PRODUCTION' status. The 'My Tasks' section has a subtitle 'Open reports from app users and incidents assigned to you. Updates live as new reports come in.' Below this is a search bar and a table of tasks.

Ticket	Issue	Station	Priority	Status	SLA	Actions
19240f1-	Screen Screen issue - screen freezing	Bida Industrial Zone	Medium	Open	—	Update Status Add Report
TXT-DEVL	Voltage anomaly detected Slow charging / power - Station reported voltage irregularities and connector fault date.	Bida Industrial Zone	Critical	Escalated	BREACHED May 21, 16:00	Update Status Add Report
TXT-DEVL	Network latency on payment terminal Network / payment - Payment authorization latency above normal threshold.	Annaba Port Station	High	Escalated	BREACHED May 20, 16:00	Update Status Add Report

At the bottom, there is a copyright notice: '© 2020 EV Charge OZ. All rights reserved.'

Figure 5.11: My Tasks Technician Personal Work Queue

The screenshot displays the SAEIG User Management module. The sidebar is the same as in previous figures. The main header shows 'SAEIG Active Context' and 'PRODUCTION' status. The 'User Management' section has a subtitle 'Create admin accounts, change roles and tenant assignments.' Below this is a search bar and a table of users.

User	Role	Privilege	Status	Tenant
Ahmed Benali ahmed.benali@sonelgaz.dz	Tenant Admin	operations_manager	active	Sonelgaz
Chiblan Abdou romanboss66@gmail.com	Tenant Admin	Standard Admin	active	SAEIG
Karim Messaoudi karim.messaoudi@saieg.dz	Tenant Admin	operations_manager	active	SAEIG
Myriem Bourou bouroumyriem@gmail.com	Super Admin	full_access	active	SAEIG
Sofiane Bensaid sofiane.bensaid@saieg.dz	Technician	Technician Default	active	SAEIG
Super Admin admin@evcharge.dz	Super Admin	full_access	active	Sonelgaz
Yacine Fodhil oy.fodhil@gmail.com	Super Admin	—	active	SAEIG
Yacine Kellou yacinekellou79@gmail.com	Super Admin	—	active	Sonelgaz
Yasmine Hadji yasmine.hadji@sonelgaz.dz	Technician	Technician Default	active	Sonelgaz

On the right, the 'Create Account' form is open, titled 'Provision a new tenant admin or technician'. It includes fields for Full Name, Email, Password, Role (Tenant Admin / Super Admin), and Privilege Template (Standard Admin, Operations Admin, Finance Admin, Technician Default, Custom).

Figure 5.12: User Management Module User List and Account Creation Form

broadcast. Operators may clear the history on demand using the dedicated button.

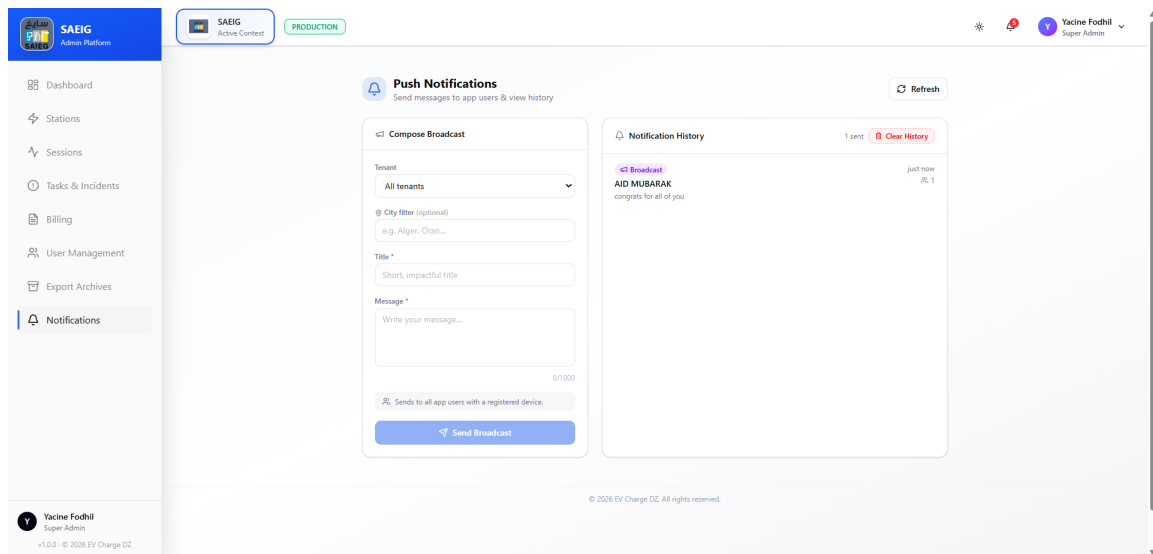


Figure 5.13: Push Notifications Module Broadcast Composer and Notification History

5.10 Data Resilience and Offline Mode

A key architectural decision is the platform’s graceful degradation strategy. The frontend checks for the presence of a backend API base URL at startup. When the backend is unreachable (network failure or server downtime), the application falls back to data persisted in the browser’s `localStorage` and displays a prominent offline-mode notification banner. This ensures operators retain read access to station lists, recent session records, and ticket data even during connectivity interruptions.

5.11 Platform Summary

5.12 Conclusion

EV Charge Manager’s implementation fulfils all ten functional requirements identified in Chapter 3 and follows the architecture designed in Chapter 4. The eight operational modules dashboard, station management, session monitoring, billing, maintenance tickets, user management, push notifications, and offline resilience collectively deliver a production-ready charging management solution adapted to Algeria’s EV infrastructure context. The platform provides a complete, locally adapted response to the gap identified in the introduction.

Chapter 6 extends the platform’s scope by presenting the EVCharge Android mobile application, the driver-facing companion that shares the same backend and database as the operator dashboard described in this chapter.

Table 5.2: EV Charge Manager Platform Specification Summary

Aspect	Detail
Platform Type	Multi-tenant SaaS web application
Target Market	Algeria (Sonelgaz and SAEIG operators)
Charging Protocol	OCPP 1.6 [8]
User Roles	super_admin, tenant_admin, technician
Granular Privileges	13 privilege types across 5 templates
Authentication	Email / password + OTP (2FA) + JWT [13]
Supported Connectors	CCS2, CHAdeMO, Type 2
Payment Methods	RFID card, CIB, e-payment
Billing Currency	Algerian Dinar (DZD)
Frontend	React 18 + TypeScript + Vite [14, 16]
Backend	Node.js + Express.js [17]
Database	MySQL 8 [18]
Map	Leaflet / OpenStreetMap
Offline Resilience	localStorage fallback with graceful degradation

Chapter 6

EVCharge : The Mobile Companion Application

6.1 Motivation and Objectives

While EV Charge Manager provides network operators with a comprehensive web dashboard for station management, session monitoring, and billing, the EV driver requires a dedicated mobile interface. The EVCharge Android application was developed as the driver-facing complement to the operator platform, closing the loop between infrastructure management and end-user experience.

The application gives drivers the ability to locate nearby charging stations on an interactive map, obtain turn-by-turn navigation directions to a selected station, report station problems directly to operators, and receive real-time push notifications about station status changes and tariff updates. All interactions are synchronised with the operator dashboard through a shared back-end infrastructure, ensuring that data visible to operators and data accessible to drivers remain consistent at all times.

6.2 Technology Stack

The application is developed in Kotlin targeting Android API 26 (Android 8.0 Oreo) as the minimum supported version and API 34 (Android 14) as the target. Kotlin is Google's officially recommended language for Android development, offering null safety, coroutine support, and full interoperability with the Android SDK.

The user interface is built entirely with Jetpack Compose and Material 3, Google's modern declarative UI toolkit for Android. Jetpack Compose eliminates XML layout files by defining UI components as composable functions that react automatically to state changes a paradigm well suited to the real-time nature of the application.

Table 6.1 summarises the complete technology stack used in the EVCharge mobile applica-

tion.

Table 6.1: EVCharge Mobile Application Technology Stack

Component	Technology / Version
Language	Kotlin (JVM target 11)
UI Framework	Jetpack Compose + Material 3 (BOM 2024.09.00)
Minimum SDK	Android 8.0 Oreo (API 26)
Target SDK	Android 14 (API 34)
Maps	OSMDroid 6.1.18 (OpenStreetMap)
HTTP Client	Retrofit 2.9.0 + OkHttp 4.12.0
Push Notifications	Firebase Cloud Messaging (FCM BOM 33.1.0)
Real-Time Updates	OkHttp WebSocket
Asynchronous I/O	Kotlin Coroutines 1.7.3

1. **Kotlin + Jetpack Compose:** Modern Android development stack. Declarative UI, native coroutines, and null safety make it well suited to a real-time, network-driven application.
2. **OSMDroid 6.1.18:** Android library for rendering OpenStreetMap tiles. Free, open-source, and consistent with the Leaflet/OpenStreetMap mapping used in the dashboard.
3. **Retrofit 2 + OkHttp 4:** Industry-standard Android HTTP client stack. Retrofit provides a type-safe REST interface; OkHttp handles connection pooling, timeouts, and WebSocket connections.
4. **Firebase Cloud Messaging:** Google’s cross-platform push messaging service, used to deliver targeted notifications from the operator backend to registered driver devices.

6.3 Application Architecture

The application follows the Repository pattern, a standard Android architecture that separates data-access logic from UI code. Each domain area authentication, stations, sessions, routes, and tickets has a dedicated repository class that encapsulates all network calls and data transformation. ViewModels expose observable state to the Compose UI layer, which reacts declaratively to state changes without polling.

The data flow is: Compose screens observe ViewModel state, ViewModels call repository methods, repositories call Retrofit API endpoints or the WebSocket manager, and responses propagate back as Kotlin Flow or Result objects. All network operations run on `Dispatchers.IO`, keeping the main thread free for UI rendering.

Network configuration is centralised in `ServerConfig`, which defines the backend host, port 3001, HTTP base URL, and WebSocket URL. `ApiClient` builds the Retrofit instance with an `OkHttp` logging interceptor and a `Gson` converter. `ApiService` declares all REST endpoint signatures as suspend functions, enabling direct use from coroutines without callback boilerplate.

6.4 App-Dashboard Integration

The most important architectural characteristic of EVCharge is that it shares the same Node.js/Express backend server and the same MySQL database with the operator web dashboard. This single-backend design ensures that data created by drivers charging sessions started from the app, maintenance tickets submitted from the app is immediately visible to operators on the dashboard, and vice versa.

6.4.1 Shared Backend, Separated API Routes

The server exposes two logically separate sets of API routes, each designed for its respective client. Dashboard routes are authenticated with operator JWT tokens and subject to role-based access control. Mobile app routes use driver JWT tokens and offer a simpler, consumer-oriented interface. Table 6.2 presents the mobile-specific API surface.

6.4.2 Authentication Difference

Dashboard operators complete a two-factor authentication flow: email/password followed by a One-Time Password sent to their registered email address, after which a JWT is issued. This two-factor requirement reflects the privileged nature of operator access to station management, billing, and user data.

Driver accounts authenticate with email and password only, without OTP verification. Drivers are consumers managing their own sessions and have no access to operator data. Passwords are hashed server-side with `bcrypt` at a cost factor of 12. The resulting JWT has a 7-day validity and is stored in Android `SharedPreferences`. Every authenticated API request includes the token as an `Authorization: Bearer` header.

6.5 Interactive Station Map

The map screen is the primary interface of the application. On launch, it calls the public endpoint `GET /api/app/stations` and renders each returned station as a colour-coded marker on an OSMDroid map backed by OpenStreetMap tiles the same tile provider used by the Leaflet map in the operator dashboard.

Table 6.2: Mobile App API Routes

Method	Endpoint	Description
POST	/api/app/auth/register	Driver account creation (email, password, full name)
POST	/api/app/auth/login	Driver login — returns JWT with 7-day validity
POST	/api/app/auth/fcm-token	Register device FCM token for push notifications
GET	/api/app/stations	Public list of all stations with GPS coordinates
GET	/api/app/stations/:id	Single station details (connectors, price, status)
GET	/api/app/sessions	Driver session history (paginated)
POST	/api/app/sessions	Start a charging session at a station
PATCH	/api/app/sessions/:id	Stop an active charging session
POST	/api/app/tickets	Report a station problem as a maintenance ticket
GET	/api/app/route	ORS routing proxy — returns GeoJSON driving route

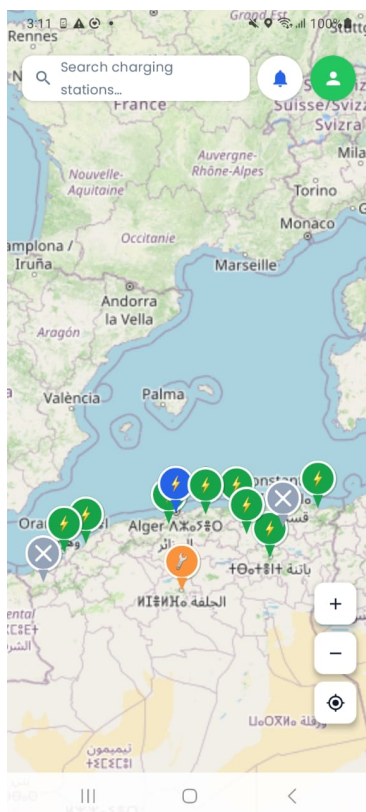
Marker colour encodes operational status: green for *available*, orange for *charging*, red for *offline* or *fault*, and grey for *maintenance*. This visual encoding is consistent with the status chip colours used in the dashboard station table, giving both operators and drivers the same semantic colour vocabulary.

Tapping a marker opens a `StationCard` bottom sheet displaying the station name, address, operational status, maximum power in kilowatts, charging price in Algerian Dinars per kilowatt-hour, supported connector types (CCS2, CHAdeMO, or Type 2), number of available connectors out of the total, and an estimated charging time. From this card the driver can launch navigation or initiate a charging session.

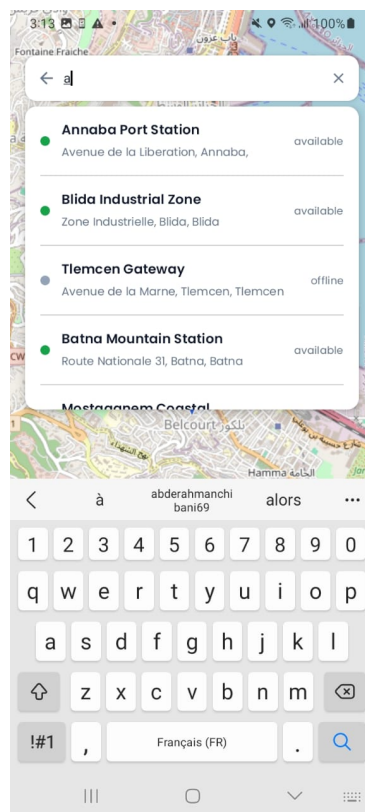
Station data is refreshed in real time by the WebSocket connection described in Section 6.7: when an operator changes a station status on the dashboard, the map marker updates on the driver device without requiring a manual refresh.

6.6 Navigation with OpenRouteService

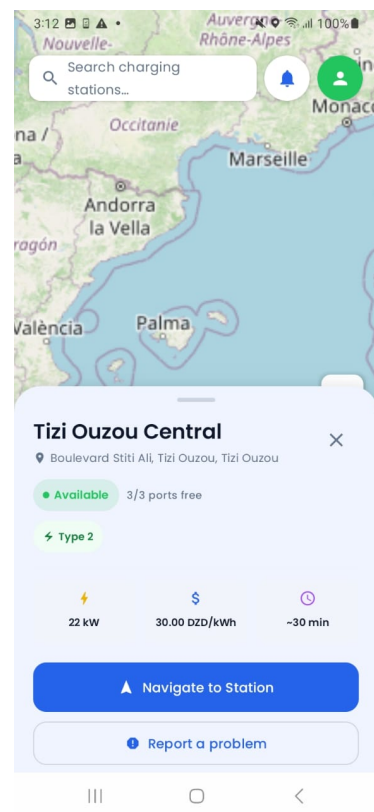
When a driver selects a station and taps `Navigate`, the application requests a driving route from OpenRouteService (ORS), an open-source routing engine built on OpenStreetMap road data. ORS computes turn-by-turn directions and returns them as a GeoJSON `FeatureCollection` containing the route geometry and a structured list of navigation steps.



(a) Station map with colour-coded markers



(b) Station search with live results



(c) StationCard bottom sheet

Figure 6.1: EVCharge — Interactive Station Map interface

6.6.1 The ORS Proxy Architecture

The ORS API is not called directly from the Android device. Instead, the request is proxied through the shared backend server at the endpoint `GET /api/app/route`, which accepts the start and end coordinates as query parameters, forwards them to the ORS API, and relays the GeoJSON response to the app. This design keeps the ORS API key server-side, preventing its exposure in the compiled APK, and allows the phone to operate even in environments where direct access to external APIs is restricted.

The proxy applies a 15-second timeout to the upstream ORS request. If the request times out or ORS returns an error, the server responds with an HTTP 502 or 504 status code, which the `RouteRepository` translates into a Kotlin `Result.failure` for the `ViewModel` to handle gracefully.

6.6.2 Route Parsing and Display

The `RouteRepository` parses the GeoJSON response into a `RouteResult` data class containing: a list of `GeoPoint` objects forming the polyline drawn on the OSMDroid map; a list of `NavStep` objects each carrying a human-readable instruction, distance in metres, duration in seconds, and an ORS maneuver type code (0 = depart, 1 = turn left, 2 = turn right, 5 = slight left, 6 = slight right, 7 = U-turn, 10 = arrive); and the total route distance and estimated duration displayed to the driver.

The `NavigationViewModel` manages step-by-step progression through the instruction list, updating the currently active instruction as the driver advances along the route. The `NavigationScreen` renders the OSMDroid map with the route polyline overlaid and a fixed instruction panel at the bottom of the screen showing the current maneuver.

The complete navigation flow, from the driver tapping `Navigate` to step-by-step route progression and error handling, is illustrated in Figure 6.3. Figure 6.2 shows the `NavigationScreen` on a real device, with the ORS route polyline drawn over Algiers and the step instruction panel active at the bottom.

6.7 Real-Time Station Updates via WebSocket

The application maintains a persistent WebSocket connection to the backend at `ws://[host]:3001/ws` from the moment the map screen is active. When a dashboard operator changes a station status for example, toggling a station into maintenance mode the server broadcasts a `station_status_update` message over this channel.

The `WebSocketManager` singleton handles the connection lifecycle using OkHttp's WebSocket API. Incoming messages are parsed from JSON and emitted to a `MutableSharedFlow`. The `MapScreen` collects this flow, which triggers a Compose recomposition that updates the affected marker colour without requiring a full data reload.

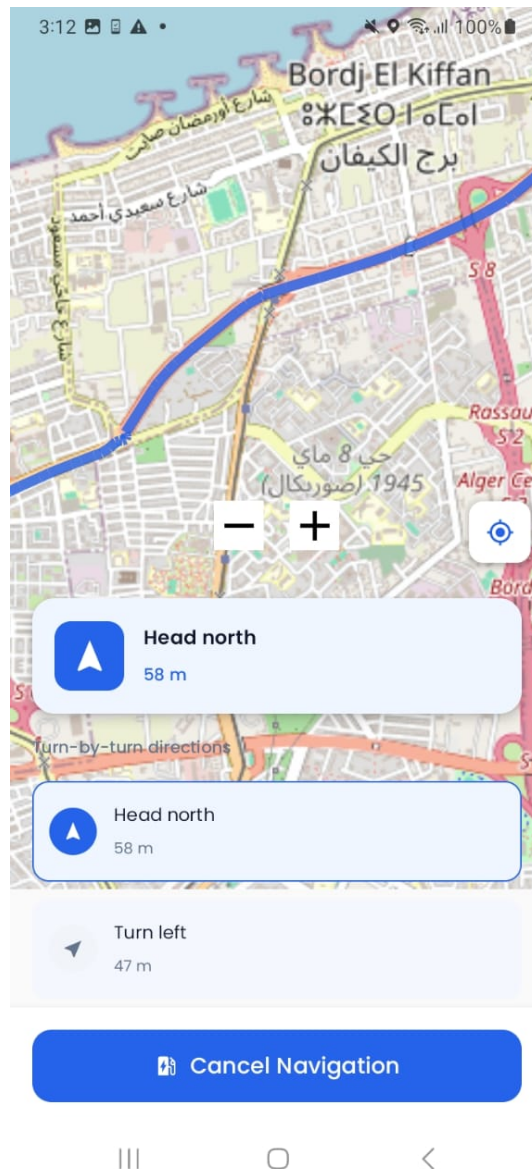


Figure 6.2: NavigationScreen — ORS route polyline and step-by-step instruction panel

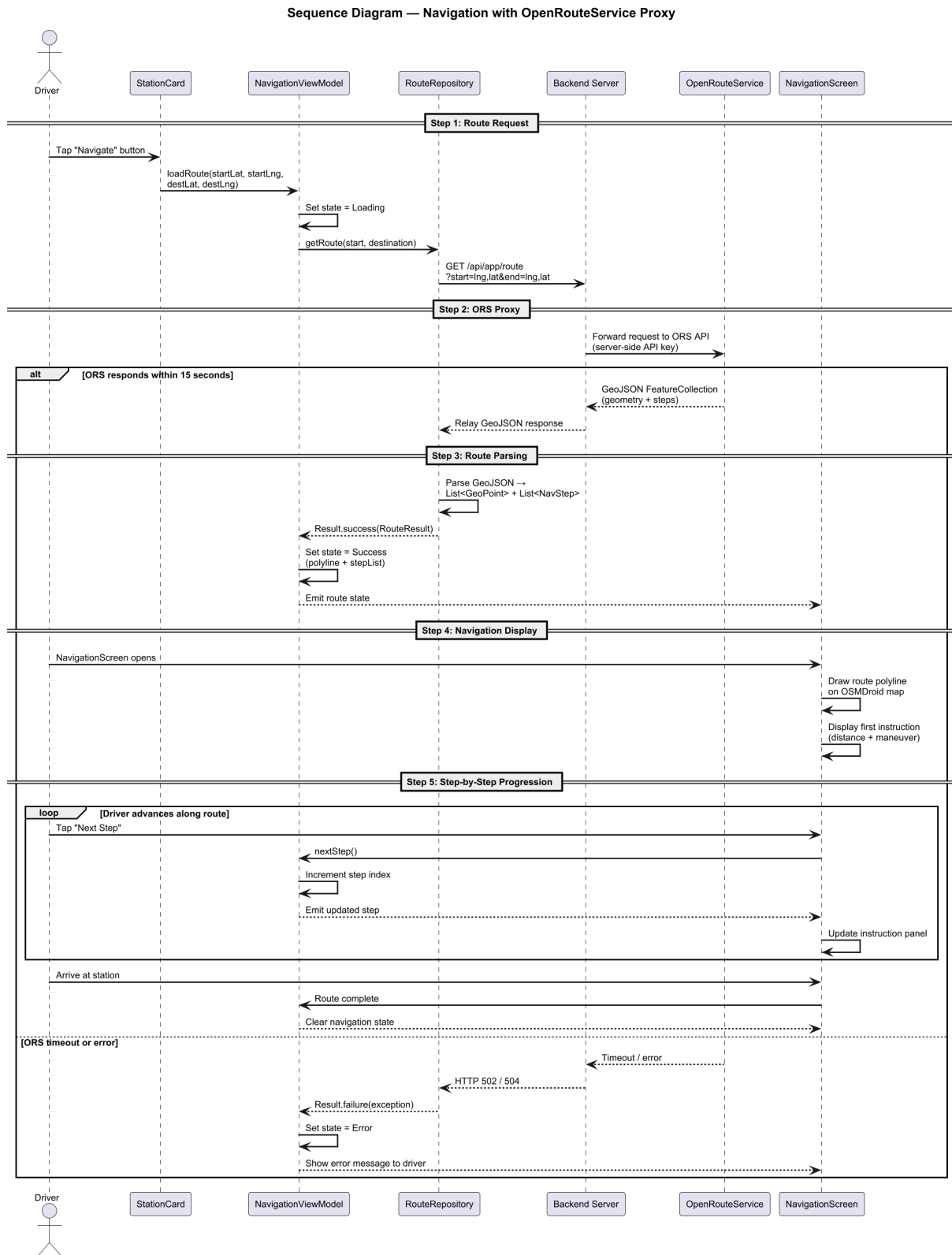


Figure 6.3: Sequence Diagram — Navigation with OpenRouteService Proxy

Auto-reconnect logic is implemented in the connection failure callback: if the WebSocket drops, the manager schedules a reconnect attempt after 5 seconds. This makes the real-time update mechanism resilient to brief network interruptions.

This WebSocket channel is shared with the operator dashboard, which uses the same server-side broadcast mechanism via the `useLiveEvents` React hook. Both the operator and the driver therefore see status changes propagate simultaneously without polling.

6.8 Firebase Cloud Messaging

Firebase Cloud Messaging (FCM) is the bridge between the operator dashboard and the driver application for asynchronous push notifications. It allows the backend to reach any registered Android device instantly, even when the application is in the background or closed.

6.8.1 Token Registration Flow

When the driver logs in, the Firebase SDK generates a device-specific FCM token. The `EvChargeMessagingService` class, which extends `FirebaseMessagingService`, intercepts the token via the `onNewToken` callback and calls `POST /api/app/auth/fcm-token` to register it with the backend. The server stores the token in the `app_users.fcm_token` database column. The callback also fires when FCM rotates the token automatically, ensuring the stored value remains current.

On the backend side, the Firebase Admin SDK is initialised with a service account. When an event requires driver notification, the `fcmHelper` module queries the database for relevant FCM tokens and dispatches messages using `admin.messaging().sendEachForMulticast()`.

6.8.2 Notification Types and Targeting

Five notification types are dispatched from the backend, each using a different targeting strategy to ensure relevance. Table 6.3 summarises the notification types, their triggers, and their targeting scope.

On the Android side, `EvChargeMessagingService` routes each incoming message to the appropriate notification channel: `station_updates` (high priority) for station status events, `tariff_updates` for price changes, and `announcements` for broadcasts and new station alerts. Each notification is also persisted to `SharedPreferences` as a JSON entry and displayed in the in-app Notifications screen, allowing drivers to review past alerts.

The complete FCM flow covering token registration and notification dispatch is illustrated in Figure 6.5. Figure 6.4 shows the in-app NotificationsScreen displaying a received broadcast announcement.



Figure 6.4: NotificationsScreen — received FCM push notifications listed chronologically

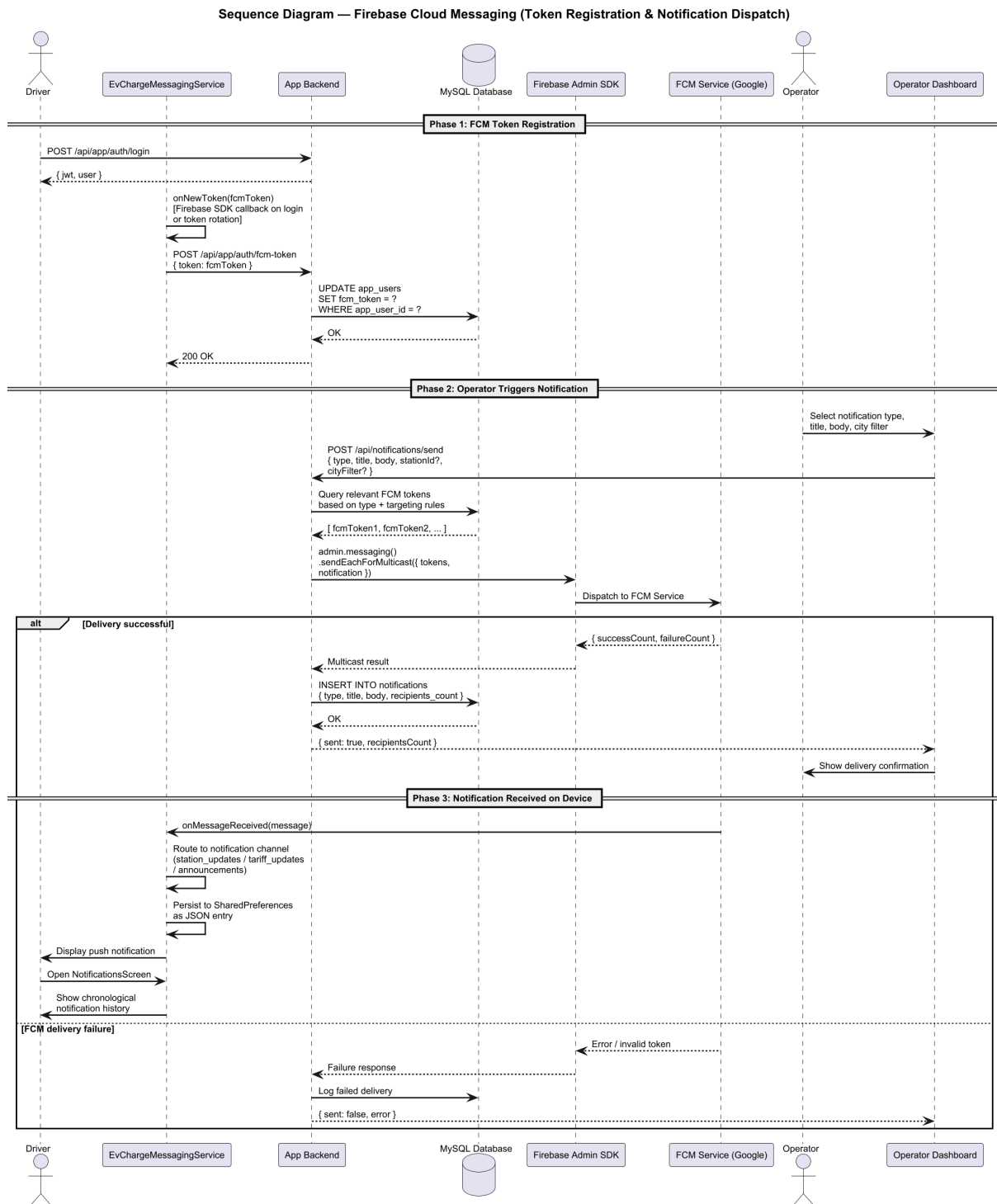


Figure 6.5: Sequence Diagram — Firebase Cloud Messaging (Token Registration & Notification Dispatch)

Table 6.3: FCM Notification Types

Type	Trigger	Target Audience
station_available	Station returns to available status	Users who charged there in the last 30 days
station_maintenance	Operator sets station to maintenance	Users who charged there in the last 30 days
tariff_change	Operator updates charging price	Users who charged there in the last 60 days
new_station	Operator adds a new station	All users in the station city
broadcast	Operator sends a manual announcement	All registered app users (optional city filter)

6.9 Charging Session Management

Drivers can start and stop charging sessions directly from the StationCard bottom sheet. Starting a session calls `POST /api/app/sessions` with the selected station identifier and the chosen payment method. The server validates station availability, creates a record in the `charging_sessions` table, and returns the session identifier, estimated cost in DZD, and a status field.

Stopping a session calls `PATCH /api/app/sessions/{session_id}` with the `action` field set to `stop`. The server computes the final duration in minutes, energy delivered in kWh, and the billed amount in DZD, closes the session record, and returns these values for display to the driver.

Session records created through the mobile app are stored in the same `charging_sessions` database table accessed by the dashboard Session Monitoring Module. Operators therefore see app-initiated sessions alongside RFID-initiated and CIB-initiated sessions in a unified view, with the source identified by the `payment_method` field.

Drivers can retrieve their session history via `GET /api/app/sessions`, which returns a paginated list of past sessions with station name, connector type, start time, duration, energy consumed, cost, and final status.

6.10 Problem Reporting

If a driver encounters a problem with a charging station, the ReportProblemSheet bottom sheet allows them to submit a maintenance ticket directly from the application. The driver selects the affected station and chooses a fault category from six options: power failure, screen issue, charger fault, system failure, network issue, or scheduled maintenance. An optional description field allows additional context.

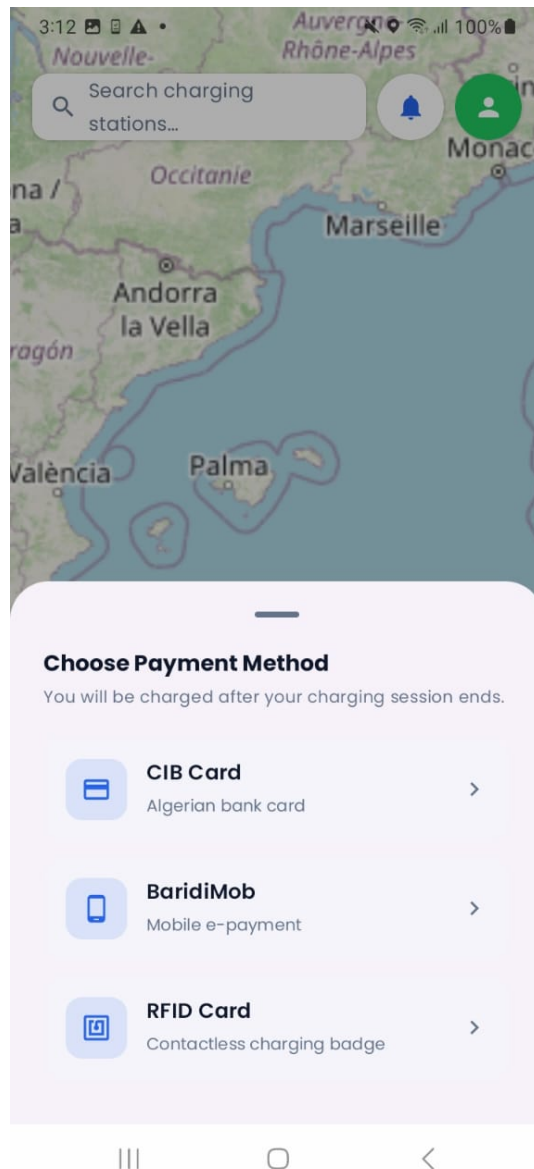


Figure 6.6: Payment method selection — CIB Card, BaridiMob, and RFID Card options

The ticket is created via POST `/api/app/tickets` and is inserted into the same tickets table used by the operator dashboard Ticket Management Module. The ticket appears immediately in the operator ticket queue with the category and an activity log entry recording its origin. Operators can then assign a technician, set an SLA deadline, and track resolution as with any operator-generated ticket.

This mechanism creates a closed feedback loop: a driver identifies a problem in the field and reports it in seconds; the operator is notified through the dashboard; a technician is assigned and resolves the issue; the station returns to available status and drivers who used that station recently receive an FCM push notification confirming availability.

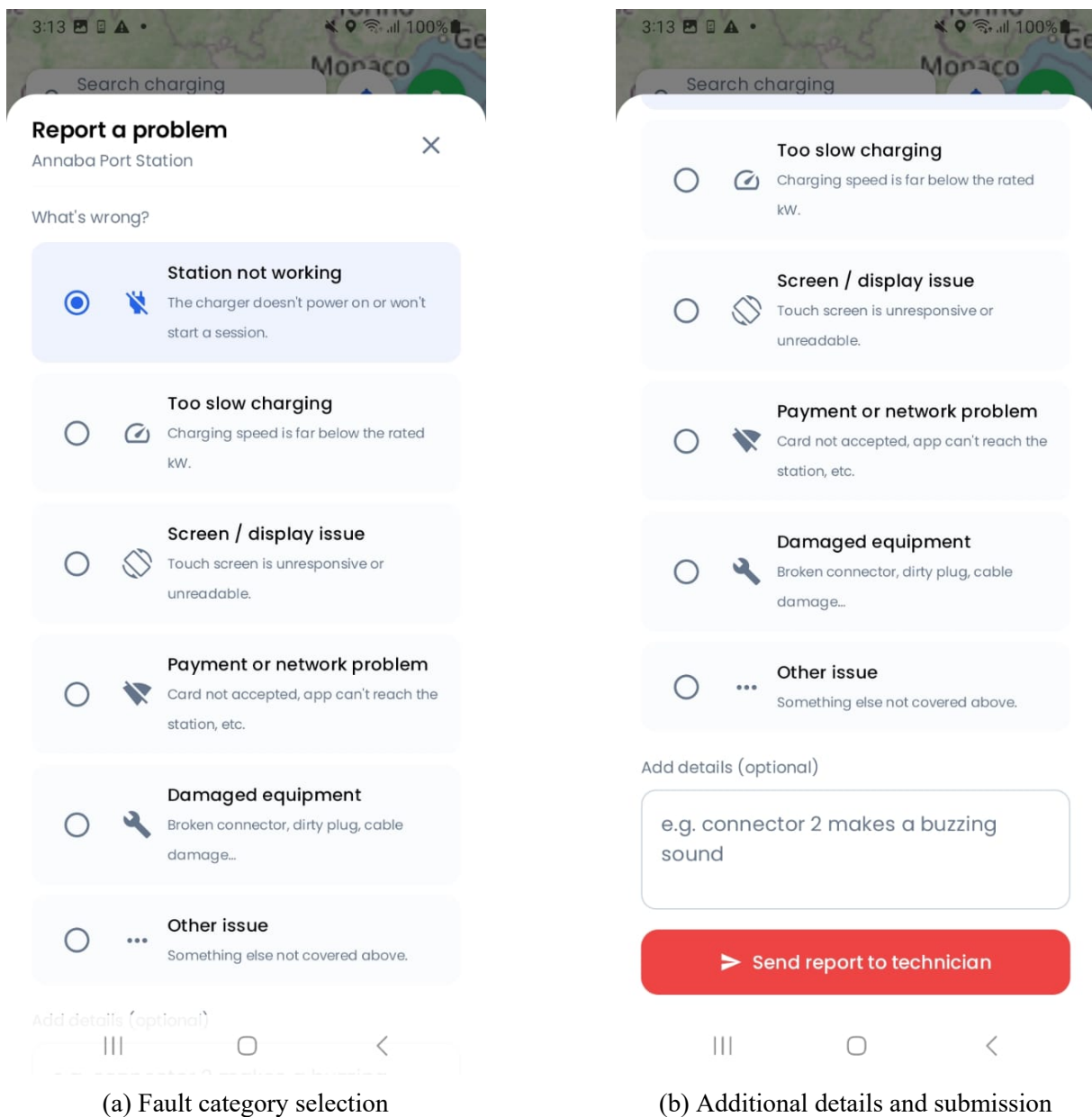


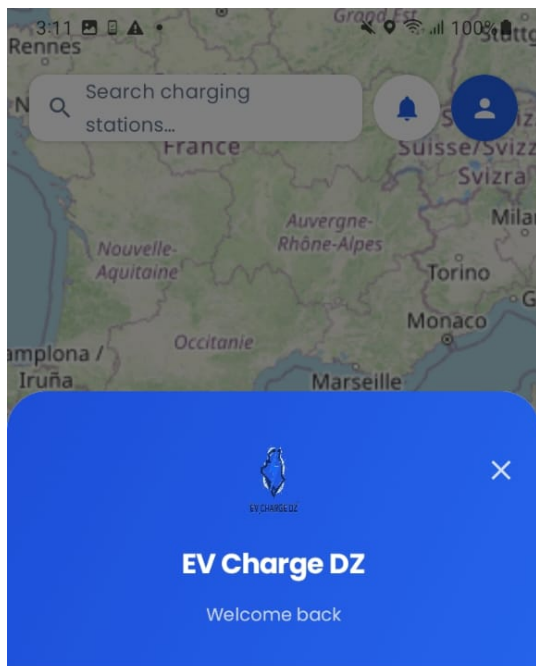
Figure 6.7: ReportProblemSheet — fault category selection and ticket submission

6.11 Application Screens Overview

Table 6.4 summarises the screens implemented in the EVCharge application, each corresponding to a source file in the `ui/screens` package.

Table 6.4: EVCharge Application Screen Summary

Screen	Description
SplashScreen	Entry point: checks for a stored JWT and routes to login or map.
LoginScreen	Email and password authentication with a registration link.
MapScreen	OSMDroid map with colour-coded station markers and real-time WebSocket updates.
StationCard	Bottom sheet showing station details with Navigate and Start Session actions.
NavigationScreen	Turn-by-turn navigation with ORS route polyline and a step instruction panel.
NotificationsScreen	Chronological list of received FCM push notifications.
ReportProblemSheet	Bottom sheet for submitting a station fault report as a maintenance ticket.



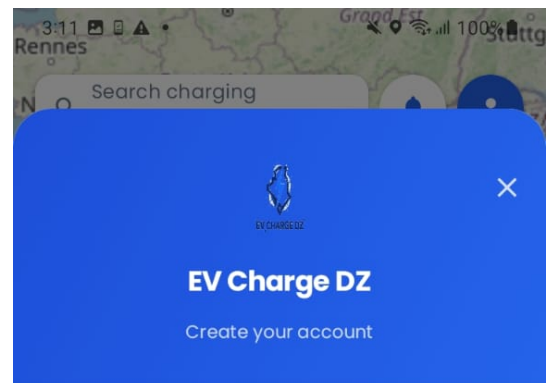
Email

Password

Don't have an account? [Sign Up](#)



(a) Login screen



Full Name

Email

Phone Number

Password

Already have an account? [Login](#)



(b) Registration screen

Figure 6.8: EVCharge — Authentication screens

6.12 Conclusion

This chapter has presented the EVCharge Android application, the driver-facing companion to the EV Charge Manager operator dashboard. Built with Kotlin and Jetpack Compose, the application covers the complete driver workflow: station discovery on an interactive map, turn-by-turn navigation via the OpenRouteService proxy, charging session initiation and termination, maintenance ticket submission, and real-time push notifications through Firebase Cloud Messaging.

A key architectural decision throughout this chapter was the shared backend: the mobile application and the web dashboard operate on the same Node.js/Express server and MySQL database, ensuring that all data sessions, tickets, and station statuses remain consistent across both interfaces without duplication or synchronisation overhead.

Three complementary communication channels support this integration: a REST API for standard data operations, a persistent WebSocket connection for instant station status propagation, and Firebase Cloud Messaging for asynchronous operator-to-driver notifications.

With the completion of this chapter, the full EV Charge Manager ecosystem is established: an operator-facing web platform and a driver-facing mobile application, working in concert over a single backend to deliver an end-to-end electric vehicle charging management solution adapted to the Algerian context.

General Conclusion and Perspectives

This work was motivated by a set of concrete gaps hindering the deployment and operation of electric vehicle charging infrastructure in Algeria. The absence of a station localisation mechanism, centralised network supervision, remote equipment control, an integrated billing system, structured maintenance tracking, and a solution built on internationally recognised open standards constituted as many obstacles to the viability and scalability of any local charging network.

In response to these problems, this thesis has presented the design and development of **EV Charge Manager**, an integrated multi-tenant web platform accompanied by complementary mobile applications, adapted to the operational and technical constraints of the Algerian context.

Each identified problem has found a concrete response in the delivered modules:

- The absence of centralised supervision was addressed by a real-time operational dashboard integrating seven key performance indicators, interactive charts, and detail panels providing a global and granular view of the station fleet.
- The inability to control equipment remotely was resolved by implementing the OCPP 1.6 protocol, enabling stations to be started, stopped, configured, and reset without physical on-site intervention.
- The absence of integrated billing was covered by an automated billing module supporting multiple payment methods RFID cards, CIB bank cards, and e-payment with per-method analytical tracking.
- The absence of an incident management system was addressed by a structured maintenance ticket system with priority levels, SLA enforcement, technician assignment, and complete intervention traceability.
- The absence of a solution built on open standards was addressed by adopting OCPP 1.6 and conducting an in-depth study of the ISO 15118 standard and the OCPP 2.0 protocol, laying the groundwork for a future evolution towards advanced features.
- The absence of a mobile interface was addressed by developing the EVCharge Android application, offering station localisation, GPS navigation, session management, maintenance ticket submission, and real-time push notifications.

On the architectural level, the multi-tenant model guarantees complete data isolation between independent operator instances, with granular role-based access control (RBAC) and thirteen fine-grained privilege flags. A graceful degradation mechanism ensures operational

continuity by switching to locally cached data when the backend is unreachable.

These achievements demonstrate the viability of an approach based on modern open-source web technologies to meet the real needs of an emerging market such as electric vehicle charging in Algeria.

Limitations

Several limitations were encountered during the realization of this project:

- First, the absence of OCPP-certified physical hardware in Algeria led to validating OCPP 1.6 communication through software simulation rather than on real equipment. Platform behaviour may differ with proprietary firmware implementations.
- Second, the platform is currently pre-configured for two tenant instances; migrating to a dynamic tenant-provisioning workflow would require additional development effort.
- Third, the billing module is not yet connected to active Algerian payment gateways such as BaridiMob and CIB online. Payment processing is currently recorded as completed without end-to-end financial settlement.
- Fourth, the EVCharge mobile application was developed and tested exclusively on the Android platform. An extension to iOS would require either a migration to a cross-platform framework or a parallel native implementation.
- Fifth, although the backend exposes a session history endpoint (GET /api/app/sessions), the corresponding mobile screen was not implemented in this version. Drivers cannot currently consult their past charging sessions from within the application.

Future Perspectives

The work carried out opens several axes of improvement and extension for future iterations of the platform:

- **Migration to OCPP 2.0.1 and native ISO 15118 integration.** Upgrading to OCPP 2.0.1 would natively enable Plug & Charge features defined by the ISO 15118 standard, implement dynamic smart charging profiles, and support bidirectional V2G energy flows, paving the way for full integration with the national electricity grid.
- **AI-powered predictive maintenance.** Integrating machine learning models exploiting ticket history and session anomalies would anticipate failures before they occur, reducing mean time to repair (MTTR) and improving overall fleet availability.
- **Smart grid integration.** Real-time communication with the national grid managed by Sonelgaz would enable demand response mechanisms, photovoltaic surplus absorption, and peak-load shaving, consistent with the capabilities described in Chapter 1 on the ISO 15118 standard.
- **Algerian payment gateway integration.** Direct connection to local payment processors, BaridiMob and CIB online, would automate invoice collection and close the financial cycle end-to-end, constituting an indispensable prerequisite for large-scale commercial

deployment.

References

- [1] João B. Santos, António M. B. Francisco, Cristiano Cabrita, João Monteiro, Alexandre Pacheco, and Pedro J. S. Cardoso. “Development and implementation of a smart charging system for electric vehicles based on the ISO 15118 standard.” In: *Energies* 17.12 (2024), p. 3045. DOI: [10.3390/en17123045](https://doi.org/10.3390/en17123045). URL: <https://doi.org/10.3390/en17123045>.
- [2] International Organization for Standardization. *ISO 15118: Road vehicles — Vehicle to grid communication interface (Parts 1–20)*. International Standard ISO 15118. Geneva, Switzerland: ISO, 2022. URL: <https://www.iso.org/standard/69113.html>.
- [3] Murat Yilmaz and Philip T. Krein. “Review of the impact of vehicle-to-grid technologies on distribution systems and utility interfaces.” In: *IEEE Transactions on Power Electronics* 28.8 (2013), pp. 3572–3591. DOI: [10.1109/TPEL.2012.2227500](https://doi.org/10.1109/TPEL.2012.2227500). URL: <https://doi.org/10.1109/TPEL.2012.2227500>.
- [4] International Electrotechnical Commission. *IEC 61851-1: Electric vehicle conductive charging system — Part 1: General requirements*. International Standard IEC 61851-1. Geneva, Switzerland: IEC, 2017. URL: <https://webstore.iec.ch/publication/33644>.
- [5] Willett Kempton and Jasna Tomić. “Vehicle-to-grid power fundamentals: Calculating capacity and net revenue.” In: *Journal of Power Sources* 144.1 (2005), pp. 268–279. DOI: [10.1016/j.jpowsour.2004.12.025](https://doi.org/10.1016/j.jpowsour.2004.12.025). URL: <https://doi.org/10.1016/j.jpowsour.2004.12.025>.
- [6] SwitchEV. *RISE-V2G: Reference implementation supporting the evolution of ISO 15118*. Open-source software. GitHub. Retrieved May 2026. 2023. URL: <https://github.com/SwitchEV/RISE-V2G>.
- [7] OpenEnergyMonitor. *Emoncms: Open-source energy monitoring web application*. Web application. Retrieved May 2026. 2024. URL: <https://emoncms.org/>.
- [8] Open Charge Alliance. *Open charge point protocol 1.6, edition 2*. Protocol Specification OCPP 1.6 Ed. 2. Open Charge Alliance, 2019. URL: <https://openchargealliance.org/protocols/open-charge-point-protocol/>.
- [9] Open Charge Alliance. *Open charge point protocol 2.0.1*. Protocol Specification OCPP 2.0.1. Open Charge Alliance, 2022. URL: <https://www.openchargealliance.org/>.

- [10] LuxmanEnergy. *OCPP 1.6 vs OCPP 2.0: A detailed comparison for EV chargers*. Online industry article. Retrieved May 2026. 2024. URL: <https://www.luxmanenergy.com/ocpp-1-6-vs-ocpp-2-0-a-detailed-comparison-for-ev-chargers/>.
- [11] AmpControl. *OCPP 1.6 vs OCPP 2.0: A comprehensive comparison*. Online industry article. Retrieved May 2026. 2024. URL: <https://www.ampcontrol.io/post/ocpp-1-6-vs-ocpp-2-0-a-comprehensive-comparison>.
- [12] International Energy Agency. *Global EV outlook 2024: Moving towards increased affordability*. Report. Paris, France: IEA, 2024. URL: <https://www.iea.org/reports/global-ev-outlook-2024>.
- [13] Michael B. Jones, John Bradley, and Nat Sakimura. *RFC 7519: JSON web token (JWT)*. Request for Comments RFC 7519. Internet Engineering Task Force, 2015. URL: <https://datatracker.ietf.org/doc/html/rfc7519>.
- [14] Meta Platforms, Inc. *React: The library for web and native user interfaces*. Official documentation. Retrieved May 2026. 2024. URL: <https://react.dev/>.
- [15] Microsoft Corporation. *TypeScript: JavaScript with syntax for types*. Official documentation. Retrieved May 2026. 2024. URL: <https://www.typescriptlang.org/>.
- [16] Evan You and Vite Contributors. *Vite: Next generation frontend tooling*. Official documentation. Retrieved May 2026. 2024. URL: <https://vite.dev/>.
- [17] OpenJS Foundation. *Node.js and Express.js: Server-side JavaScript runtime and web framework*. Official documentation. Retrieved May 2026. 2024. URL: <https://nodejs.org/>.
- [18] Oracle Corporation. *MySQL 8.0 reference manual*. Official documentation. Retrieved May 2026. 2024. URL: <https://dev.mysql.com/doc/refman/8.0/en/>.
- [19] Tailwind Labs, Inc. *Tailwind CSS: A utility-first CSS framework for rapid UI development*. Official documentation. Retrieved May 2026. 2024. URL: <https://tailwindcss.com/>.
- [20] Recharts Group. *Recharts: A composable charting library built on React components*. Official documentation. Retrieved May 2026. 2024. URL: <https://github.com/recharts/recharts>.
- [21] Remix Software, Inc. *React Router v6: Declarative routing for React applications*. Official documentation. Retrieved May 2026. 2024. URL: <https://reactrouter.com/>.
- [22] European Parliament and Council. *Regulation (EU) 2016/679 of the European Parliament and of the Council: General data protection regulation (GDPR)*. Official Journal of the European Union. 2016. URL: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.

Appendix A

Combined Use Case Diagram (All Actors)

The diagram below is the original combined use case diagram presenting all three actors — Super Admin, Tenant Admin, and Technician — within a single view. It is provided here for reference, as its size made it unsuitable for inline presentation in the body of the report.

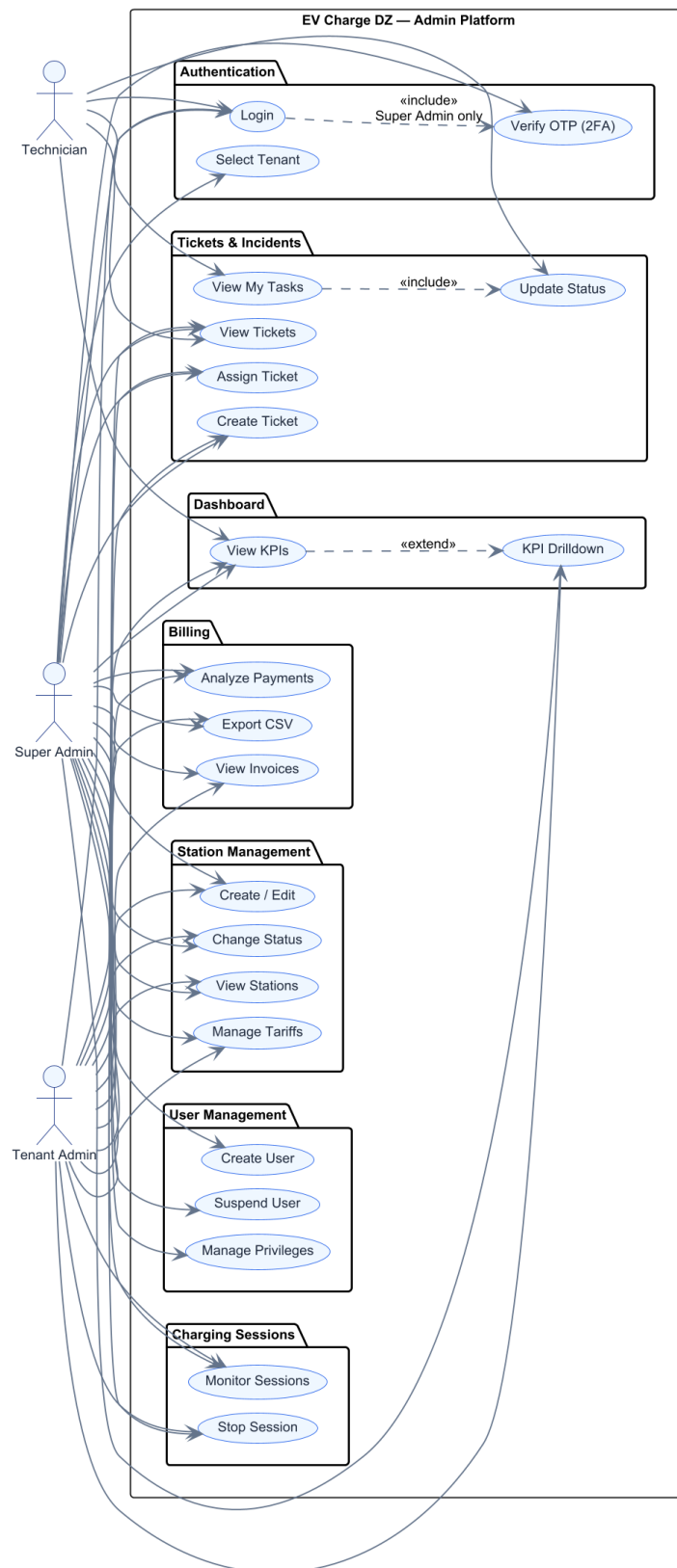


Figure A.1: Combined Use Case Diagram — All Actors

Appendix B

Complete Class Diagram (with Enumeration Types)

The diagram below reproduces the full database-oriented class diagram of EV Charge DZ, including all enumeration types (UserRole, UserStatus, StationStatus, SessionStatus, TicketPriority, TicketStatus, TicketCategory, PaymentStatus, PaymentMethod, AlertSeverity). These types are referenced as attribute domains within the core entities but carry no inter-entity relationship; they are presented here for completeness.

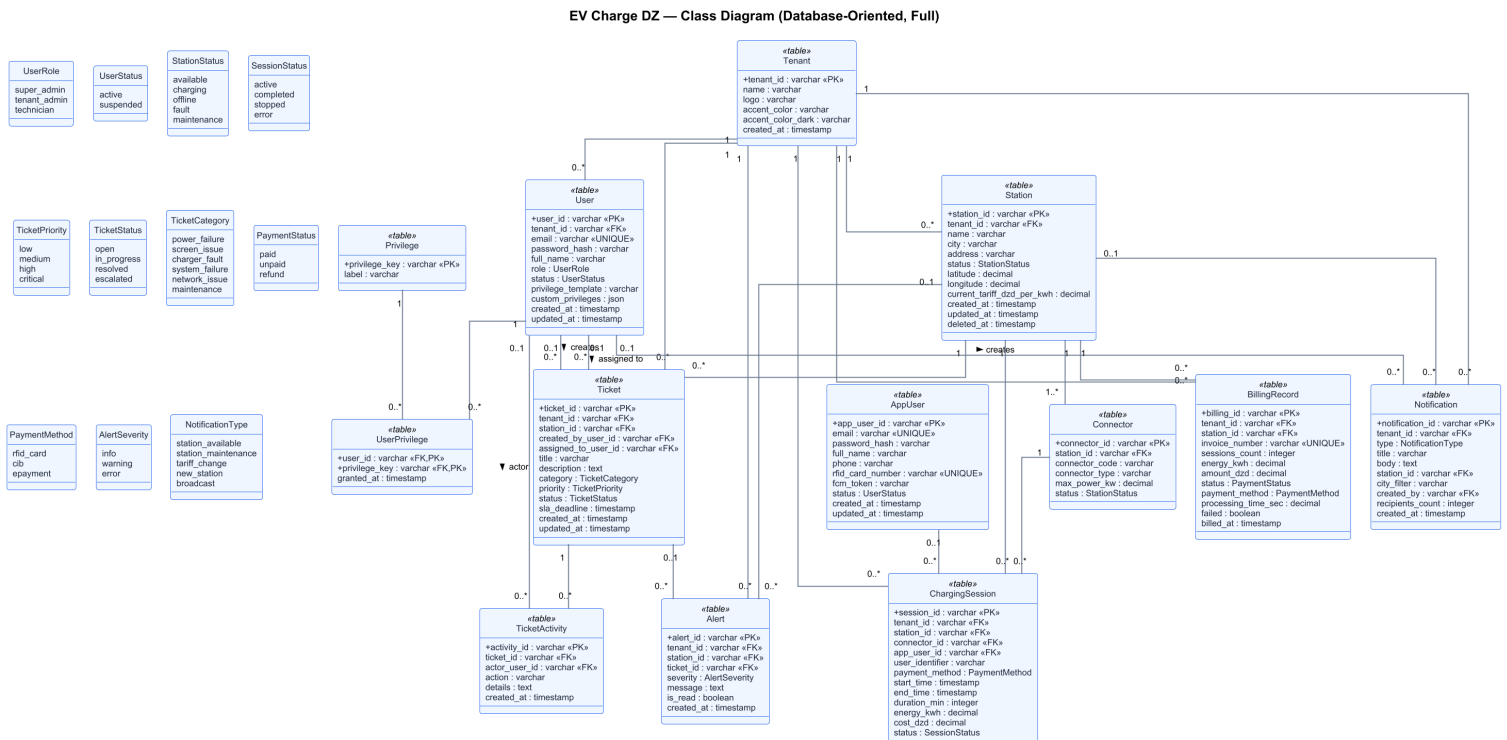


Figure B.1: Full class diagram including enumeration types